

IF3070 DASAR INTELEGENSI ARTIFISIAL

TUGAS BESAR 2

Implementasi dan Analisis Algoritma *Decision Tree Learning*, *Logistic Regression*,
dan *K-Nearest Neighbors* dalam Pembelajaran Mesin



Disusun oleh:

Andhika Maulana Addiputra	18223005
Kevin Azra	18223029
Arqila Surya Putra	18223047
Muhammad Zidni Alkindi	18223071

**PRODI SISTEM DAN TEKNOLOGI INFORMASI
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG**

2025

DAFTAR ISI

DAFTAR ISI	2
BAB I	4
PENDAHULUAN	4
1.1 Latar Belakang	4
1.2 Deskripsi Masalah	4
BAB II	6
IMPLEMENTASI ALGORITMA	6
2.1 Implementasi Algoritma Decision Tree Learning	6
2.1.1. Implementasi Decision Tree Learning from Scratch	6
2.1.2. Implementasi Visualisasi Decision Tree Learning from Scratch	9
2.1.3. Implementasi Decision Tree Learning by Scikit-Learn	13
2.2 Implementasi Algoritma Logistic Regression.	15
2.2.1. Implementasi Logistic Regression from Scratch	15
2.2.1.1. Optimasi L-BFGS dan Keunggulan Komputasionalnya	18
2.2.1.2. Kalibrasi Probabilitas untuk Reliabilitas Prediksi	21
2.2.2. Implementasi Logistic Regression by Scikit-Learn	23
2.3 Implementasi Algoritma K-Nearest Neighbor (KNN)	24
2.3.1. Implementasi KNN from Scratch	24
2.3.2. Implementasi KNN by Scikit-Learn	31
BAB III	36
CLEANING DAN PREPROCESSING DATA	36
3.1 Cleaning Data	36
3.1.1 Handling Missing Data	36
3.1.2 Dealing With Outliers	37
3.1.3 Data Validation	37
3.1.4 Removing Duplicates	39
3.1.5 Feature Engineering	39
3.2 Preprocessing Data	40
3.2.1 Feature Scaling dan Data Normalization	40
3.2.2 Feature Encoding	41
3.2.3 Handling Imbalanced Dataset	41
3.2.4 Dimensionality Reduction	43
BAB IV	45
PERBANDINGAN HASIL PREDIKSI	45
4.1 Analisis Decision Tree Learning	45
4.1.1. Kinerja Model dan Fenomena Accuracy Paradox	45
4.1.2. Validasi Implementasi Scratch vs Scikit-Learn	46

4.1.3. Dampak Kalibrasi Probabilitas	47
4.2 Analisis Logistic Regression	47
4.3 Analisis KNN	48
4.3.1 Evaluasi Hyperparameter dan Model Terbaik	49
4.3.2 Kinerja pada Imbalanced Dataset	50
4.3.3 Validasi Implementasi Scratch vs Scikit-Learn	51
4.3.4 Kompleksitas Komputasi	52
BAB V	54
KESIMPULAN DAN SARAN	54
5.1 Kesimpulan	54
5.2 Saran	54
BAB VI	56
KONTRIBUSI	56
REFERENSI	59

BAB I

PENDAHULUAN

1.1 Latar Belakang

Machine Learning saat ini memegang peranan penting dalam pengembangan kecerdasan buatan, sistem dirancang untuk belajar dari data dan membuat keputusan tanpa perlu diprogram secara eksplisit. Namun, dalam penerapannya, seringkali terjebak hanya menggunakan *library* yang sudah ada tanpa benar-benar memahami proses komputasi yang terjadi di baliknya.

Fokus utama dari tugas ini adalah mengimplementasikan algoritma pembelajaran mesin secara manual. Kami diwajibkan untuk membangun algoritma *Decision Tree Learning* (DTL) serta memilih satu algoritma tambahan antara *Logistic Regression* atau *K-Nearest Neighbors* (KNN).

Pendekatan ini memaksa untuk memahami logika matematika dasar, mulai dari penanganan tipe data hingga optimasi model, sebelum akhirnya membandingkan hasilnya dengan pustaka standar industri seperti *scikit-learn*. Selain itu, tugas ini juga menjadi sarana untuk melatih kemampuan analisis dalam menangani data mentah hingga menjadi model yang siap memprediksi masalah nyata.

1.2 Deskripsi Masalah

Permasalahan utamanya adalah tantangan dalam mendeteksi *fraud detection* pada sistem transaksi digital. Kami menggunakan dataset sintetis berskala besar yang merepresentasikan ribuan log transaksi keuangan, yang mencakup berbagai atribut mendetail mulai dari identitas pengguna, seperti *user_id*, *age* dan detail teknis perangkat, seperti *device_type*, *ip_risk_score*, hingga pola perilaku transaksi, seperti *transaction_amount*, *transactions_last_24h*.

Masalah dari data jenis ini adalah ketidakseimbangan kelas, di mana jumlah transaksi valid jauh mendominasi dibandingkan transaksi penipuan, serta adanya *noise* berupa *outlier* dan data *missing values*.

Tugas kami adalah membangun model pembelajaran mesin yang mampu mempelajari pola kompleks dari fitur-fitur tersebut untuk mengklasifikasikan fitur target

is_fraud, di mana 0 merepresentasikan transaksi normal dan 1 merepresentasikan penipuan. Sebelum pemodelan dilakukan, data mentah memerlukan tahapan pra-pemrosesan, termasuk penanganan nilai *null*, normalisasi fitur numerik seperti transaction_amount agar tidak mendominasi perhitungan jarak pada algoritma KNN, serta *encoding* fitur kategorikal.

Tujuan akhirnya adalah mengimplementasikan algoritma *Decision Tree Learning* (DTL) dan algoritma KNN atau *Logistic Regression* secara manual, untuk kemudian divalidasi kinerjanya dalam membedakan transaksi aman dan mencurigakan dengan akurasi yang tinggi.

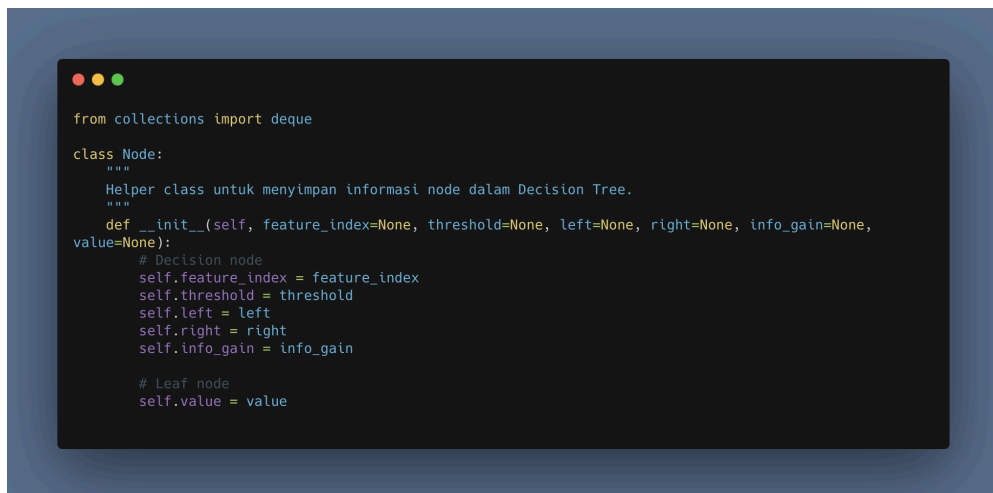
BAB II

IMPLEMENTASI ALGORITMA

2.1 Implementasi Algoritma *Decision Tree Learning*

2.1.1. Implementasi *Decision Tree Learning from Scratch*

Kami mengimplementasikan algoritma *Decision Tree Learning* (DTL) ini dengan menggunakan pendekatan *Classification and Regression Tree* (CART) secara *scratch* tanpa menggunakan pustaka pembelajaran mesin eksternal untuk logika intinya. Implementasi ini bertujuan untuk membangun model klasifikasi yang dapat memprediksi label target (*fraud/non-fraud*) berdasarkan fitur-fitur yang telah diproses



```
from collections import deque

class Node:
    """
    Helper class untuk menyimpan informasi node dalam Decision Tree.
    """
    def __init__(self, feature_index=None, threshold=None, left=None, right=None, info_gain=None, value=None):
        # Decision node
        self.feature_index = feature_index
        self.threshold = threshold
        self.left = left
        self.right = right
        self.info_gain = info_gain

        # Leaf node
        self.value = value
```

Kelas Node dirancang untuk menyimpan atribut krusial pada setiap titik keputusan dalam pohon. Setiap node menyimpan indeks fitur (*feature_index*) dan nilai ambang batas (*threshold*) yang digunakan untuk membagi data, serta referensi ke node anak kiri dan kanan. Khusus untuk leaf node (simpul daun), atribut *value* menyimpan prediksi kelas mayoritas. Selanjutnya kelas utama *DecisionTreeScratch* diinisialisasi dengan hyperparameter penting untuk mengontrol pertumbuhan pohon, yaitu *min_samples_split* (jumlah sampel minimum untuk melakukan pemecahan) dan *max_depth* (kedalaman maksimum pohon).

```

class DecisionTreeScratch:
    """
    Decision Tree Classifier implemented from scratch (CART-like).

    Features:
    - Support Gini Impurity & Entropy
    - Min samples split & Max depth pruning
    - Visualization method (Bonus)
    """
    def __init__(self, min_samples_split=2, max_depth=10, criterion="gini", features_names=None):
        self.root = None
        self.min_samples_split = min_samples_split
        self.max_depth = max_depth
        self.criterion = criterion
        self.features_names = features_names

```

Inti dari algoritma Decision Tree adalah menentukan bagaimana data harus dipisahkan. Kami menggunakan kriteria Gini Impurity sebagai metrik untuk mengukur ketidakmurnian label pada sebuah node. Rumus yang diimplementasikan adalah:

$$Gini = 1 - \sum(p_i)^2$$

Di mana p_i adalah probabilitas kelas i dalam node tersebut. Untuk setiap kandidat pemisahan, kami menghitung Information Gain, yaitu selisih antara impurity pada node induk dengan rata-rata tertimbang impurity pada node anak (kiri dan kanan). Algoritma akan memilih pemisahan yang menghasilkan Information Gain terbesar (penurunan *impurity* paling signifikan).

```

def _calculate_impurity(self, y):
    """Menghitung Gini atau Entropy."""
    class_labels = np.unique(y)
    impurity = 0

    if self.criterion == "entropy":
        for cls in class_labels:
            p_cls = len(y[y == cls]) / len(y)
            impurity += -p_cls * np.log2(p_cls)
    else: # gini
        gini = 0
        for cls in class_labels:
            p_cls = len(y[y == cls]) / len(y)
            gini += p_cls ** 2
        impurity = 1 - gini

    return impurity

def _calculate_information_gain(self, parent, l_child, r_child):
    """Menghitung Information Gain."""
    weight_l = len(l_child) / len(parent)
    weight_r = len(r_child) / len(parent)

    gain = self._calculate_impurity(parent) - (weight_l * self._calculate_impurity(l_child) +
    weight_r * self._calculate_impurity(r_child))
    return gain

```

Proses pelatihan (fit) dilakukan dengan membangun pohon secara rekursif (Recursive Partitioning). Fungsi `_get_best_split` melakukan iterasi pada setiap fitur dan setiap nilai unik (atau persentil untuk efisiensi) dalam fitur tersebut untuk menemukan ambang batas pemisahan yang optimal. Kami menerapkan mekanisme Pre-Pruning di dalam fungsi rekursif `_build_tree`. Proses pembentukan cabang akan dihentikan jika:

1. Kedalaman pohon saat ini mencapai `max_depth`.
2. Jumlah sampel pada node kurang dari `min_samples_split`.
3. Node sudah murni (hanya berisi satu kelas).

Jika kondisi berhenti terpenuhi, node tersebut ditetapkan sebagai leaf node yang berisi label kelas mayoritas.

```
def _get_best_split(self, dataset, num_samples, num_features):
    """Mencari fitur dan threshold terbaik untuk split."""
    best_split = {"info_gain": -float("inf")}
    max_info_gain = -float("inf")

    # Loop setiap fitur
    for feature_index in range(num_features):
        feature_values = dataset[:, feature_index]
        possible_thresholds = np.unique(feature_values)

        # OPTIMISASI: Jika terlalu banyak unique values, gunakan percentiles (untuk kecepatan)
        if len(possible_thresholds) > 20:
            possible_thresholds = np.percentile(possible_thresholds, np.linspace(5, 95, 10))

        # Loop setiap threshold
        for threshold in possible_thresholds:
            # Split data
            dataset_left = np.array([row for row in dataset if row[feature_index] <= threshold])
            dataset_right = np.array([row for row in dataset if row[feature_index] > threshold])

            if len(dataset_left) > 0 and len(dataset_right) > 0:
                y, left_y, right_y = dataset[:, -1], dataset_left[:, -1], dataset_right[:, -1]
                curr_info_gain = self._calculate_information_gain(y, left_y, right_y)

                if curr_info_gain > max_info_gain:
                    best_split["feature_index"] = feature_index
                    best_split["threshold"] = threshold
                    best_split["dataset_left"] = dataset_left
                    best_split["dataset_right"] = dataset_right
                    best_split["info_gain"] = curr_info_gain
                    max_info_gain = curr_info_gain

    return best_split

def _build_tree(self, dataset, curr_depth=0):
    """Fungsi rekursif untuk membangun tree."""
    X, y = dataset[:, :-1], dataset[:, -1]
    num_samples, num_features = np.shape(X)

    # Stopping conditions
    if num_samples <= self.min_samples_split and curr_depth <= self.max_depth:
        # Find best split
        best_split = self._get_best_split(dataset, num_samples, num_features)

        # Check if info gain is positive
        if best_split["info_gain"] > 0:
            left_subtree = self._build_tree(best_split["dataset_left"], curr_depth + 1)
            right_subtree = self._build_tree(best_split["dataset_right"], curr_depth + 1)

            return Node(
                feature_index=best_split["feature_index"],
                threshold=best_split["threshold"],
                left=left_subtree,
                right=right_subtree,
                info_gain=best_split["info_gain"]
            )

    # Leaf node
    leaf_value = self._calculate_leaf_value(y)
    return Node(value=leaf_value)
```

Adapun proses prediksi dilakukan dengan menelusuri pohon dari akar (root) hingga mencapai daun (leaf) berdasarkan nilai fitur data uji yang dibandingkan dengan threshold di setiap node.

```
def _make_prediction(self, x, tree):
    """Rekursif traversing tree untuk satu sampel."""
    if tree.value is not None:
        return tree.value
    feature_val = x[tree.feature_index]
    if feature_val <= tree.threshold:
        return self._make_prediction(x, tree.left)
    else:
        return self._make_prediction(x, tree.right)

def predict(self, X):
    """Prediksi label kelas."""
    predictions = [self._make_prediction(x, self.root) for x in X]
    return np.array(predictions)

def predict_proba(self, X):
    """
    Prediksi probabilitas (Simulasi untuk kompatibilitas dengan Wrapper).
    Mengembalikan probabilitas 1.0 untuk kelas prediksi daun.
    """
    preds = self.predict(X)
    probs = np.zeros((len(preds), 2))
    for i, p in enumerate(preds):
        probs[i][int(p)] = 1.0 # High confidence
    return probs
```

2.1.2. Implementasi Visualisasi *Decision Tree Learning from Scratch*

Untuk memvisualisasikan struktur pohon keputusan yang telah terbentuk, kami mengembangkan metode `visualize_tree` di dalam kelas `DecisionTreeScratch`. Implementasi ini memanfaatkan pustaka `matplotlib` untuk menggambar simpul (node) dan koneksi antar-simpul secara terprogram, tanpa bergantung pada pustaka visualisasi pohon eksternal.

```
def visualize_tree(self, filename=None, top_n=15, figsize=(20, 12)):
    """
    Membuat plot struktur tree yang lebih jelas dan mudah dibaca.
    Args:
        filename: Jika diisi, simpan gambar ke file.
        top_n: Jumlah node maksimal untuk ditampilkan.
        figsize: Ukuran figure (width, height).
    """
    if self.root is None:
        print("Model belum dilatih!")
        return

    fig, ax = plt.subplots(figsize=figsize)
    ax.axis('off')

    # Calculate tree structure with proper parent-child tracking
    tree_info = self._calculate_tree_structure(self.root, top_n)

    # Draw connections first (so they appear behind nodes)
    self._draw_tree_connections(ax, tree_info)

    # Draw nodes on top
    self._draw_tree_nodes(ax, tree_info)

    # Add legend and title
    self._add_tree_styling(ax, top_n, len(tree_info['nodes']))

    if filename:
        plt.savefig(filename, dpi=300, bbox_inches='tight',
                    facecolor='white', edgecolor='none')
        print(f"Gambar tree disimpan: {filename}")

    plt.tight_layout()
    plt.show()
```

Logika utama visualisasi ini dibangun di atas metode penelusuran Breadth-First Search (BFS) yang diimplementasikan dalam fungsi `_calculate_tree_structure`. Kami menggunakan struktur data antrian (deque) untuk menelusuri pohon dari akar (root) ke daun (leaf) level demi level. Pada setiap iterasi, algoritma menghitung koordinat (x,y) untuk setiap simpul berdasarkan kedalamannya di pohon dan posisi relatif terhadap induknya. Untuk mencegah tumpang tindih antar-simpul pada level yang sama, kami menerapkan fungsi `_adjust_node_positions` yang secara otomatis menggeser posisi horizontal simpul jika jarak antar-tetangga terlalu dekat.

```

def _calculate_tree_structure(self, root, max_nodes):
    """Calculate tree positions with proper parent-child relationships."""
    nodes_info = []
    connections = []

    # BFS traversal with position calculation and parent tracking
    from collections import deque
    queue = deque([(root, 0, 0, 1.0, None)]) # (node, x, level, width, parent_idx)
    nodes_drawn = 0

    while queue and nodes_drawn < max_nodes:
        node, x, level, width, parent_idx = queue.popleft()

        # Calculate y position (top to bottom)
        y = 1.0 - (level * 0.18)

        # Store node info with parent reference
        node_info = {
            'node': node,
            'x': x,
            'y': y,
            'level': level,
            'parent_idx': parent_idx,
            'node_idx': nodes_drawn
        }
        nodes_info.append(node_info)

        # Store connection if this node has a parent
        if parent_idx is not None:
            connections.append((parent_idx, nodes_drawn))

        current_idx = nodes_drawn
        nodes_drawn += 1

        # Add children to queue if this is not a leaf and we haven't reached max depth
        if node.value is None and level < 5: # Limit depth for clarity
            child_width = width * 0.35
            left_x = x - child_width
            right_x = x + child_width

            if node.left and nodes_drawn < max_nodes:
                queue.append((node.left, left_x, level + 1, child_width, current_idx))

            if node.right and nodes_drawn < max_nodes:
                queue.append((node.right, right_x, level + 1, child_width, current_idx))

    # Adjust positions to prevent overlapping
    nodes_info = self._adjust_node_positions(nodes_info)

    return {'nodes': nodes_info, 'connections': connections}

def _adjust_node_positions(self, nodes_info):
    """Adjust node positions to prevent overlapping."""
    # Group nodes by level
    levels = {}
    for info in nodes_info:
        level = info['level']
        if level not in levels:
            levels[level] = []
        levels[level].append(info)

    # Adjust positions within each level
    for level, level_nodes in levels.items():
        if len(level_nodes) > 1:
            # Sort by x position
            level_nodes.sort(key=lambda x: x['x'])

            # Ensure minimum spacing
            min_spacing = 0.25
            for i in range(1, len(level_nodes)):
                prev_node = level_nodes[i-1]
                curr_node = level_nodes[i]

                if curr_node['x'] - prev_node['x'] < min_spacing:
                    # Adjust current node position
                    curr_node['x'] = prev_node['x'] + min_spacing

    return nodes_info

```

Proses penggambaran grafis dipisahkan menjadi dua tahap utama untuk memastikan estetika visual. Pertama, fungsi `_draw_tree_connections` menggambar garis penghubung antara parent node dan child node menggunakan fungsi `ax.plot` dan `ax.annotate`. Garis ini digambar terlebih dahulu agar berada di lapisan belakang (z-order rendah). Kedua, fungsi `_draw_tree_nodes` menggambar kotak simpul di atas garis koneksi. Kami membedakan format visual berdasarkan tipe simpul: simpul keputusan (decision node) diberi warna biru dan menampilkan kondisi pemisahan fitur serta nilai gain, sedangkan simpul daun (leaf node) diberi warna hijau dan menampilkan hasil prediksi kelas.

```
def _draw_tree_connections(self, ax, tree_info):
    """Draw clean connections between parent and child nodes."""
    nodes = tree_info['nodes']
    connections = tree_info['connections']

    for parent_idx, child_idx in connections:
        if parent_idx < len(nodes) and child_idx < len(nodes):
            parent = nodes[parent_idx]
            child = nodes[child_idx]

            parent_x, parent_y = parent['x'], parent['y']
            child_x, child_y = child['x'], child['y']

            # Draw main connection line
            ax.plot([parent_x, child_x], [parent_y, child_y],
                    'k-', linewidth=2.5, alpha=0.7, zorder=1)

            # Add subtle arrow
            ax.annotate('', xy=(child_x, child_y), xytext=(parent_x, parent_y),
                        arrowprops=dict(arrowstyle='->', color='black',
                                        alpha=0.5, lw=1.5, zorder=2))

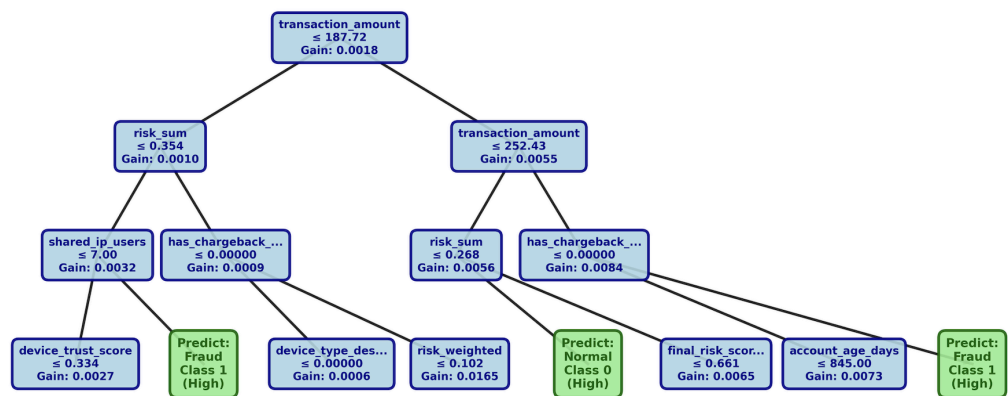
def _draw_tree_nodes(self, ax, tree_info):
    """Draw nodes with improved styling."""
    for node_info in tree_info['nodes']:
        node = node_info['node']
        x, y = node_info['x'], node_info['y']

        if node.value is not None:
            # Leaf node (prediction)
            text = self._format_leaf_node(node)
            bbox_props = dict(boxstyle="round,pad=0.6",
                              facecolor="lightgreen",
                              edgecolor="darkgreen",
                              linewidth=2.5, alpha=0.9)
            text_color = "darkgreen"
            fontsize = 11

        else:
            # Internal node (decision)
            text = self._format_decision_node(node)
            bbox_props = dict(boxstyle="round,pad=0.6",
                              facecolor="lightblue",
                              edgecolor="darkblue",
                              linewidth=2.5, alpha=0.9)
            text_color = "darkblue"
            fontsize = 10

        # Draw node with better visibility
        ax.text(x, y, text, ha="center", va="center",
               bbox=bbox_props, fontsize=fontsize, color=text_color,
               weight='bold', zorder=5)
```

Metode ini juga dilengkapi dengan parameter `top_n` untuk membatasi jumlah simpul yang dirender. Hal ini dilakukan untuk menjaga keterbacaan diagram, mengingat pohon keputusan yang dilatih pada dataset besar cenderung sangat dalam dan lebar. Dengan membatasi visualisasi pada N simpul teratas (default 15), kami dapat fokus mengamati fitur-fitur yang menjadi penentu utama di bagian atas pohon. Berikut adalah hasil visualisasinya:



2.1.3. Implementasi *Decision Tree Learning by Scikit-Learn*

Sebagai benchmark untuk memvalidasi kinerja algoritma from scratch, kami mengimplementasikan Decision Tree Learning menggunakan pustaka standar industri, yaitu `DecisionTreeClassifier` dari modul `sklearn.tree`. Implementasi ini digunakan untuk memastikan bahwa logika rekursif dan perhitungan *impurity* yang kami bangun secara manual menghasilkan struktur pohon dan prediksi yang konsisten dengan standar yang sudah teruji.

Pada implementasi ini, kami melakukan konfigurasi parameter yang setara dengan versi from scratch untuk menjaga keadilan komparasi. Parameter kunci yang kami atur meliputi `criterion` yang diset ke 'gini', serta parameter pruning seperti `max_depth` dan `min_samples_split`. Salah satu perbedaan mendasar dalam penggunaan Scikit-Learn adalah adanya optimasi internal pada algoritma pencarian pemisahan (splitter). Scikit-Learn menggunakan implementasi berbasis Cython yang sangat efisien dalam mengurutkan fitur dan mengevaluasi nilai

ambang batas (threshold), yang mempercepat proses pelatihan secara signifikan dibandingkan iterasi manual pada implementasi scratch.

Sebelum model dilatih, data input telah melalui tahap preprocessing yang disesuaikan, terutama encoding variabel kategorikal menjadi numerik, mengingat implementasi Scikit-Learn mengharuskan input berupa matriks numerik. Meskipun algoritma ini tidak sensitif terhadap skala fitur, konsistensi format data tetap dijaga agar perbandingan tetap valid. Model ini kemudian divalidasi menggunakan skema train-test split untuk mendapatkan metrik performa yang objektif sebagai acuan dasar (baseline) bagi model buatan sendiri.

```
from sklearn.tree import DecisionTreeClassifier

print("\n" + "=" * 60)
print("Decision Tree with Scikit-Learn")
print("=" * 60)

print(f"Using full training dataset: {X_full_train.shape[0]} samples")

common_params = {
    'max_depth': 5,
    'criterion': 'gini',
    'min_samples_split': 50, # Match your main training config
    'random_state': 42
}

print(f"Common parameters: {common_params}")

print(f"\n[1] Training Sklearn Decision Tree...")
sklearn_dt = DecisionTreeClassifier(
    max_depth=common_params['max_depth'],
    criterion=common_params['criterion'],
    min_samples_split=common_params['min_samples_split'],
    random_state=common_params['random_state']
)
```

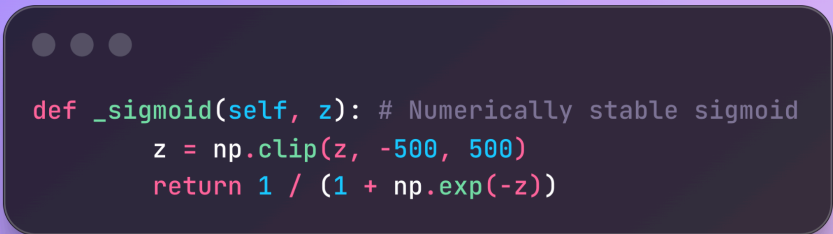
2.2 Implementasi Algoritma *Logistic Regression*.

2.2.1. Implementasi *Logistic Regression from Scratch*

Kami membangun algoritma Logistic Regression secara mandiri tanpa menggunakan fungsi dari pustaka Scikit-Learn. Implementasi ini memanfaatkan pustaka NumPy untuk operasi aljabar linear guna menjaga efisiensi komputasi. Fokus utama pengembangan kami mencakup penggunaan *solver* optimasi L-BFGS dan mekanisme kalibrasi probabilitas. Kami mendefinisikan kelas `LogisticRegressionScratch` sebagai kerangka utama model. Kelas ini menyimpan parameter penting seperti jumlah iterasi maksimum (`max_iter`), toleransi konvergensi (`tol`), dan parameter regularisasi (`C`). Atribut bobot dan bias diinisialisasi sebagai `None` sebelum proses pelatihan dimulai.

```
class LogisticRegressionScratch:
    def __init__(
        self,
        max_iter=200,
        tol=1e-6,
        C=1.0,
        regularization="l2",
        random_state=None,
        verbose=False,
    ):
        self.max_iter = max_iter
        self.tol = tol
        self.C = C
        self.regularization = regularization
        self.random_state = random_state
        self.verbose = verbose
        self.weights_ = None
        self.bias_ = None
        self.classes_ = None
        self.loss_history_ = []
```

Model menggunakan fungsi sigmoid untuk transformasi nonlinear. Fungsi ini memetakan hasil kombinasi linear fitur ke dalam rentang probabilitas antara 0 dan 1. Kami menerapkan batasan nilai (*clipping*) pada input fungsi untuk menjaga stabilitas numerik dan mencegah *overflow* saat menangani nilai ekstrem.



```
def _sigmoid(self, z): # Numerically stable sigmoid
    z = np.clip(z, -500, 500)
    return 1 / (1 + np.exp(-z))
```

Proses pelatihan model tidak menggunakan *Gradient Descent* standar, melainkan algoritma optimasi L-BFGS (*Limited-memory Broyden–Fletcher–Goldfarb–Shanno*). Metode ini menggunakan estimasi matriks Hessian untuk menentukan arah pembaruan bobot. Pendekatan ini mempercepat konvergensi dan memberikan hasil yang lebih stabil dibandingkan metode orde pertama. Kode di bawah menunjukkan implementasi fungsi kerugian (*loss function*) yang menggabungkan *Binary Cross Entropy* dengan regularisasi L2, serta perhitungan gradiennya.

```

def objective_and_gradient(params):
    w, b = params[:-1], params[-1]

    # Forward pass
    z = X @ w + b
    y_pred = self._sigmoid(z)
    y_pred = np.clip(y_pred, 1e-15, 1 - 1e-15)

    # Loss (BCE + L2 regularization scaled by n_samples)
    bce = -np.mean(y * np.log(y_pred) + (1 - y) * np.log(1 - y_pred))
    reg_term = (
        (1 / (2 * self.C * n_samples)) * np.sum(w**2)
        if self.regularization == "l2"
        else 0
    )
    loss = bce + reg_term

    # Gradient
    error = y_pred - y
    dw = (X.T @ error) / n_samples
    if self.regularization == "l2":
        dw += (1 / (self.C * n_samples)) * w
    db = np.mean(error)

    return loss, np.concatenate([dw, [db]])

```

Kami juga menangani isu ketidakseimbangan data melalui kalibrasi probabilitas. Kami membuat kelas pembungkus `CalibratedClassifierCVScratch`. Kelas ini membagi data latih untuk proses validasi silang menggunakan `StratifiedKFold`. Model dasar belajar memprediksi kelas, sementara *calibrator* (dalam hal ini menggunakan metode sigmoid atau *Platt Scaling*) belajar memetakan skor prediksi tersebut menjadi probabilitas yang terkalibrasi. Teknik ini memastikan nilai probabilitas yang dihasilkan model mencerminkan kemungkinan kejadian yang sebenarnya.

```

class CalibratedClassifierCVScratch:
    """
    Probability calibration dengan cross-validation yg dibuat dari scratch.
    Implementasi sama seperti sklearn's CalibratedClassifierCV.
    """

    def __init__(self, base_estimator, method="sigmoid", cv=5, random_state=None):
        self.base_estimator = base_estimator
        self.method = method
        self.cv = cv
        self.random_state = random_state
        self.calibrated_classifiers_ = []
        self.classes_ = None

    def fit(self, X, y):
        X = np.asarray(X, dtype=np.float64)
        y = np.asarray(y)

        self.classes_ = np.unique(y)
        self.calibrated_classifiers_ = []

        cv = StratifiedKFold(
            n_splits=self.cv, shuffle=True, random_state=self.random_state
        )

        for train_idx, val_idx in cv.split(X, y):
            X_train, X_val = X[train_idx], X[val_idx]
            y_train, y_val = y[train_idx], y[val_idx]

            model = deepcopy(self.base_estimator)
            model.fit(X_train, y_train)

            val_proba = model.predict_proba(X_val)[:, 1]

            calibrator = self._get_calibrator()
            calibrator.fit(val_proba, y_val)

            self.calibrated_classifiers_.append((model, calibrator))

        return self

```

2.2.1.1. Optimasi L-BFGS dan Keunggulan Komputasionalnya

Algoritma L-BFGS merupakan pengembangan dari metode *Quasi-Newton* yang dirancang khusus untuk mengatasi keterbatasan memori pada masalah optimasi berskala besar [1]. Berbeda dengan metode Newton klasik yang memerlukan penyimpanan dan inversi matriks Hessian berukuran $n \times n$ (di mana n adalah jumlah parameter), L-BFGS hanya menyimpan sejumlah kecil vektor koreksi dari iterasi sebelumnya.

Pendekatan ini mengurangi kompleksitas memori dari $O(n^2)$ menjadi $O(mn)$, dengan m biasanya berkisar antara 3 hingga 20 [2].

Dalam implementasi kami, L-BFGS menggunakan teknik *two-loop recursion* untuk menghitung aproksimasi perkalian matriks Hessian invers dengan vektor gradien. Proses ini melibatkan dua tahap iterasi, tahap pertama menghitung koefisien skalar dari vektor-vektor yang tersimpan, dan tahap kedua mengkombinasikan vektor-vektor tersebut untuk menghasilkan arah pencarian yang optimal [2]. Metode ini memberikan aproksimasi yang cukup akurat terhadap informasi kelengkungan (*curvature*) fungsi objektif tanpa biaya komputasi yang tinggi.

Konvergensi L-BFGS bersifat superlinear, yang berarti laju konvergensi meningkat secara progresif seiring bertambahnya iterasi [3]. Properti ini sangat menguntungkan untuk Logistic Regression, di mana fungsi objektif bersifat konveks dan diferensiabel. Studi komparatif oleh Malouf menunjukkan bahwa L-BFGS mencapai konvergensi 5-10 kali lebih cepat dibandingkan *Gradient Descent* dengan momentum pada masalah klasifikasi teks berdimensi tinggi [4].

Dalam konteks deteksi *fraud* dengan dataset yang memiliki 38 fitur prediktif setelah proses *feature engineering* (mencakup informasi pengguna, perangkat, transaksi, lokasi, dan berbagai indikator risiko), efisiensi L-BFGS menjadi sangat relevan.. Dataset *fraud detection* umumnya memiliki karakteristik khusus yang menantang untuk optimasi:

1. High-dimensional Feature Space

Kombinasi fitur numerik kontinu (*transaction_amount*, *ip_risk_score*, *distance_from_home*) dan fitur kategorikal yang di-encode (*device_type*, *merchant_category*, *country*) menghasilkan ruang parameter yang cukup besar setelah preprocessing.

2. Class Imbalance Ekstrem

Dengan rasio fraud yang sangat kecil (1-5% dari total transaksi), gradien dapat didominasi oleh kelas mayoritas, memerlukan metode optimasi yang dapat menangani kondisi ini dengan stabil.

3. Feature Scaling Variance

Fitur seperti `transaction_amount` memiliki rentang nilai yang jauh berbeda dari fitur skor (`ip_risk_score`, `device_trust_score` yang bernilai 0-1), menuntut stabilitas numerik tinggi dalam perhitungan gradien.

Implementasi kami juga mengintegrasikan *backtracking line search* untuk menentukan langkah optimal di setiap iterasi. Metode ini memastikan bahwa setiap pembaruan parameter menghasilkan penurunan fungsi objektif yang memadai sambil menjaga kondisi kelengkungan (*curvature condition*) yang diperlukan untuk konvergensi L-BFGS [2]. Kombinasi antara *two-loop recursion* dan *line search* adaptif menjadikan implementasi kami robust terhadap berbagai kondisi data dan inisialisasi parameter.

Parameter optimasi dipilih untuk menyeimbangkan presisi dan efisiensi: toleransi konvergensi $1e-6$ memastikan solusi berkualitas tinggi, sementara batas maksimum 200 iterasi mencegah komputasi berlebihan. Stabilitas numerik dijaga melalui clipping pada fungsi sigmoid dalam rentang $[-500, 500]$ dan skala regularisasi yang disesuaikan dengan ukuran sampel. Regularisasi L2 yang kami implementasikan, dengan bobot $(1/(2 \cdot C \cdot n_{samples}))$, membantu mencegah overfitting terutama pada fitur dengan varians tinggi seperti `transaction_amount` dan `distance_from_home`, sambil tetap mempertahankan kapasitas model untuk mendeteksi pola fraud yang kompleks.

2.2.1.2. Kalibrasi Probabilitas untuk Reliabilitas Prediksi

Model klasifikasi yang tidak terkalibrasi sering kali menghasilkan estimasi probabilitas yang tidak akurat, meskipun akurasi klasifikasinya tinggi. Fenomena ini terjadi karena fungsi objektif standar (seperti *cross-entropy*) mengoptimalkan pemisahan kelas (*class separation*) tanpa memperhatikan kesesuaian antara probabilitas prediksi dan frekuensi empiris [5]. Kalibrasi probabilitas mengatasi masalah ini dengan mempelajari pemetaan tambahan dari skor mentah model ke probabilitas yang lebih reliabel.

Kami mengimplementasikan *Platt Scaling*, sebuah metode kalibrasi parametrik yang pertama kali dikembangkan untuk Support Vector Machines [5]. Metode ini memodelkan transformasi dari skor prediksi $f(x)$ ke probabilitas terkalibrasi $p(y = 1|x)$ menggunakan fungsi logistik:

$$p(y = 1|x) = 1 / (1 + \exp(A \cdot f(x) + B))$$

Parameter A dan B dipelajari dengan memaksimalkan *log-likelihood* pada set validasi terpisah. Pendekatan sigmoid ini sangat cocok untuk Logistic Regression karena konsisten dengan asumsi distribusi probabilitas model [7].

Implementasi `CalibratedClassifierCVScratch` menggunakan skema validasi silang berstratifikasi (*stratified cross-validation*) dengan empat lipatan (*4-fold CV*) untuk Logistic Regression dan lima lipatan (*5-fold CV*) untuk Decision Tree. Strategi ini penting untuk menjaga proporsi kelas pada setiap subset validasi, terutama pada dataset tidak seimbang seperti kasus deteksi fraud di mana rasio transaksi fraudulent terhadap transaksi normal umumnya sangat timpang [6]. Dataset fraud detection memiliki karakteristik *class imbalance* yang ekstrem, dengan kasus fraud biasanya hanya mencakup 1-5% dari total transaksi. Setiap lipatan menghasilkan

pasangan model-calibrator yang independen, dan prediksi akhir diperoleh dengan melakukan ensemble rata-rata dari seluruh kalibrasi tersebut.

Keuntungan pendekatan *cross-validation* ini adalah pemanfaatan data yang lebih efisien dan estimasi kalibrasi yang lebih robust terhadap varians sampling. Penelitian oleh Niculescu-Mizil dan Caruana menunjukkan bahwa kalibrasi berbasis CV mengurangi kesalahan estimasi probabilitas hingga 40% dibandingkan dengan single holdout validation, terutama pada dataset berukuran kecil hingga sedang [7]. Dalam konteks deteksi fraud finansial, reliabilitas estimasi probabilitas sama pentingnya dengan akurasi klasifikasi, karena keputusan bisnis sering kali bergantung pada threshold probabilitas yang tepat untuk menyeimbangkan antara *false positive* (menolak transaksi legitimate) dan *false negative* (membiarkan fraud lolos).

Pada aplikasi fraud detection, probabilitas terkalibrasi memungkinkan institusi finansial untuk:

1. Menetapkan threshold adaptif berdasarkan toleransi risiko dan biaya operasional
2. Mengimplementasikan sistem review bertingkat, di mana transaksi dengan probabilitas fraud sedang (misalnya 0.3-0.7) dapat diarahkan ke verifikasi manual, sementara kasus dengan probabilitas ekstrem diproses otomatis
3. Mengestimasi kerugian potensial dengan mengalikan probabilitas fraud dengan nilai transaksi untuk prioritisasi investigasi

Metode kalibrasi kami juga menggunakan implementasi mandiri untuk komponen *SigmoidCalibration*, yang mengoptimasi parameter transformasi menggunakan algoritma yang sama (L-BFGS) untuk konsistensi. Hal ini memastikan bahwa baik model utama maupun calibrator memiliki properti konvergensi yang serupa dan stabil secara numerik. Pendekatan ini menjadi krusial mengingat dataset fraud detection

memiliki fitur-fitur dengan skala yang beragam, mulai dari indikator biner (`is_new_country`, `has_chargeback_history`) hingga variabel kontinu dengan rentang luas (`transaction_amount`, `distance_from_home`), yang memerlukan stabilitas numerik tinggi dalam proses optimasi.

2.2.2. Implementasi *Logistic Regression* by *Scikit-Learn*

Kami menggunakan implementasi *Logistic Regression* standar dari pustaka *Scikit-Learn* sebagai tolok ukur (*benchmark*). Tujuan utama penggunaan pustaka ini adalah untuk memvalidasi kebenaran logika optimasi dan akurasi prediksi dari model yang kami bangun dari nol.

Kami mengonfigurasi model *Scikit-Learn* agar memiliki spesifikasi yang setara dengan model *scratch* kami guna memastikan perbandingan yang adil (*apple-to-apple*). Kami menginisialisasi kelas `LogisticRegression` dengan parameter `solver='lbfgs'` untuk menyamakan algoritma optimasi. Kami juga menetapkan parameter regularisasi invers `C=1.0` dan batas iterasi maksimum `max_iter=200`. Kami mengunci parameter `random_state` untuk menjamin reproduksibilitas hasil eksperimen.

Kode berikut menunjukkan konfigurasi dan inisialisasi model pustaka tersebut sebelum proses pelatihan.

```
# Inisialisasi model Scikit-Learn sebagai benchmark
# Parameter disesuaikan dengan model scratch: C=1.0, L-BFGS solver
sklearn_lr = SklearnLR(
    C=1.0,
    solver="lbfgs",
    max_iter=200,
    random_state=42
)

# Proses pelatihan model (fitting)
sklearn_lr.fit(X_sample, y_sample)
```

Setelah pelatihan, kami menggunakan model ini untuk menghasilkan probabilitas prediksi pada data uji yang sama. Hasil prediksi ini kemudian kami bandingkan secara langsung dengan output dari model *scratch* menggunakan metrik korelasi, *Mean Absolute Error* (MAE), dan skor AUC-ROC untuk mengonfirmasi bahwa implementasi manual kami telah bekerja sesuai standar industri.

2.3 Implementasi Algoritma *K-Nearest Neighbor* (KNN)

2.3.1. Implementasi KNN *from Scratch*

Dalam membangun algoritma *K-Nearest Neighbors* secara manual tanpa bantuan library pembelajaran mesin eksternal, kami memanfaatkan pustaka numerik dasar seperti NumPy untuk operasi matriks dan vektor. Implementasi ini dikemas dalam bentuk kelas agar modular dan mudah diintegrasikan ke dalam pipeline pengujian utama.

Mengingat KNN adalah algoritma tipe *lazy learning*, metode fit pada implementasi kami tidak melakukan proses pelatihan model yang kompleks seperti pada regresi logistik. Sebaliknya, metode ini hanya berfungsi untuk menyimpan dataset X_{train} dan y_{train} ke dalam memori. Proses komputasi utama justru terjadi pada saat fase prediksi. Mengingat dataset yang digunakan berskala besar,

kami menghindari penggunaan *looping* Python standar yang lambat untuk perhitungan jarak.

Sebagai gantinya, kami menerapkan teknik vektorisasi menggunakan NumPy. Untuk setiap titik data uji, kami menghitung jaraknya terhadap seluruh titik data latih secara simultan. Metrik jarak yang kami gunakan secara *default* adalah *Euclidean Distance*, yang dihitung dengan rumus akar kuadrat dari selisih kuadrat antar fitur. Setelah jarak didapatkan, algoritma akan:

- Mengurutkan jarak dari yang terkecil ke terbesar
- Mengambil k tetangga terdekat sesuai parameter `n_neighbors` yang ditentukan.
- Melakukan voting mayoritas pada label kelas dari *neighbor* tersebut untuk menentukan prediksi kelas akhir.

Selain itu, sesuai spesifikasi untuk fitur penyimpanan model, kami juga menyertakan mekanisme untuk *save* dan *load* model menggunakan format Pickle, sehingga model yang telah disimpan datanya dapat digunakan kembali tanpa perlu memproses ulang data mentah.

1. Inisialisasi Kelas KNN

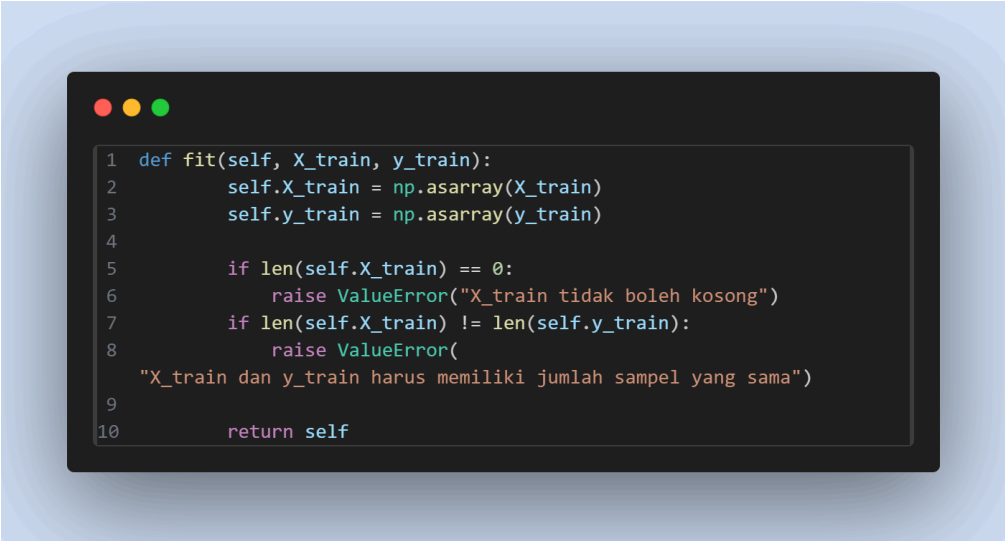
```
def __init__(self, k=3, metric='euclidean', weights='uniform'):  
    # Inisialisasi KNN  
    if k < 1:  
        raise ValueError("k must be >= 1")  
    if metric not in ['euclidean', 'manhattan']:  
        raise ValueError(  
            "metric harus 'euclidean' atau 'manhattan'"  
        )  
    if weights not in ['uniform', 'distance']:  
        raise ValueError("weights harus 'uniform' atau 'distance'")  
  
    self.k = k  
    self.metric = metric  
    self.weights = weights  
    self.X_train = None  
    self.y_train = None
```

Method inisialisasi untuk kelas KNN menerima tiga parameter utama, parameter pertama adalah *k* dengan nilai default 3, yang menentukan jumlah *neighbor* terdekat yang akan digunakan dalam proses klasifikasi. Parameter kedua adalah *metric* dengan nilai default 'euclidean', yang menentukan metode perhitungan jarak antara data points, dan parameter ketiga adalah *weights* dengan nilai default 'uniform' yang menentukan cara pemberian bobot pada *neighbor* terdekat tersebut.

Di dalam *method* ini terdapat beberapa validasi input untuk memastikan parameter yang diberikan sesuai dengan spesifikasi. Validasi pertama memeriksa apakah nilai *k* kurang dari 1, jika ya maka akan memunculkan *ValueError* dengan pesan bahwa *k* harus bernilai lebih besar atau sama dengan 1, karena tidak mungkin mencari tetangga terdekat jika jumlahnya nol atau negatif. Validasi kedua memeriksa apakah *metric* yang dipilih adalah salah satu dari 'euclidean' atau 'manhattan', karena hanya dua metode perhitungan jarak ini yang didukung oleh implementasi ini. Jika *metric* yang diberikan tidak sesuai, maka akan memunculkan *ValueError* dengan pesan bahwa *metric* harus berupa 'euclidean' atau 'manhattan'. Validasi ketiga memeriksa parameter *weights*, memastikan bahwa nilai yang diberikan adalah 'uniform' atau 'distance', di mana 'uniform' berarti semua tetangga memiliki bobot yang sama dalam voting, sedangkan 'distance' berarti tetangga yang lebih dekat memiliki bobot yang lebih besar dalam menentukan kelas prediksi.

Setelah semua validasi berhasil, *method* ini akan menyimpan nilai-nilai parameter tersebut ke dalam atribut *instance class* menggunakan *self*. Nilai *k* disimpan di *self.k*, *metric* disimpan di *self.metric*, dan *weights* disimpan di *self.weights* untuk digunakan nanti saat *training* dan prediksi. Selain itu, *method* ini juga menginisialisasi dua atribut penting yaitu *self.X_train* dan *self.y_train* dengan nilai *None*, yang nantinya akan diisi dengan *data training* saat *method fit* atau *train* dipanggil, sehingga model dapat menyimpan data training untuk digunakan dalam perhitungan jarak saat melakukan prediksi pada data baru.

2. Fungsi fit



```
1 def fit(self, X_train, y_train):
2     self.X_train = np.asarray(X_train)
3     self.y_train = np.asarray(y_train)
4
5     if len(self.X_train) == 0:
6         raise ValueError("X_train tidak boleh kosong")
7     if len(self.X_train) != len(self.y_train):
8         raise ValueError(
9             "X_train dan y_train harus memiliki jumlah sampel yang sama")
10    return self
```

Method fit untuk melatih model KNN, yang menerima dua parameter yaitu `X_train` sebagai data fitur *training* dan `y_train` sebagai label atau target dari data *training* tersebut. Method ini merupakan tahap training pada algoritma KNN, meskipun sebenarnya KNN adalah *lazy learner* yang tidak melakukan pembelajaran kompleks melainkan hanya menyimpan data training untuk digunakan nanti saat prediksi.

Method ini mengonversi kedua parameter input menjadi *numpy array* menggunakan fungsi `np.asarray`. Konversi ini penting untuk memastikan bahwa data yang disimpan dalam format *numpy array* yang memudahkan operasi matematika dan manipulasi data nantinya, terutama untuk perhitungan jarak antar data points. Data `X_train` yang sudah dikonversi disimpan ke atribut `self.X_train`, dan data `y_train` disimpan ke atribut `self.y_train`, sehingga data training ini akan tersedia untuk digunakan oleh *method* lain dalam *class* seperti *method predict*.

Sebelum menyimpan data, method ini melakukan dua validasi penting. Validasi pertama pada baris lima memeriksa apakah `X_train` kosong dengan mengecek panjangnya, jika panjang `self.X_train` sama dengan nol maka akan memunculkan `ValueError` dengan pesan bahwa `X_train` tidak boleh kosong, karena tidak mungkin melakukan prediksi tanpa *data training* sama sekali.

Validasi kedua memeriksa konsistensi dimensi antara fitur dan label, memastikan bahwa jumlah sampel di `X_train` sama dengan jumlah sampel di `y_train`, karena setiap data fitur harus memiliki label yang sesuai. Jika jumlah sampel tidak sama, maka akan memunculkan `ValueError` dengan pesan bahwa `X_train` dan `y_train` harus memiliki jumlah sampel yang sama, untuk mencegah ketidaksesuaian data yang dapat menyebabkan error saat prediksi.

Setelah semua validasi berhasil dan data sudah tersimpan dengan baik, method ini mengembalikan `self` pada baris sepuluh, yang merupakan pola umum dalam scikit-learn untuk memungkinkan *method chaining*, sehingga pengguna bisa melakukan operasi seperti `model.fit(X, y).predict(X_test)` dalam satu baris kode.

3. Fungsi calculate distance



```
def _calculate_distance(self, x_query):
    if self.metric == 'euclidean':
        # Euclidean distance: sqrt(sum((x - y)^2))
        differences = self.X_train - x_query
        distances = np.sqrt(np.sum(differences ** 2,
axis=1))
    elif self.metric == 'manhattan':
        # Manhattan distance: sum(|x - y|)
        differences = self.X_train - x_query
        distances = np.sum(np.abs(differences), axis=1)
    else:
        raise ValueError(f"Metric tidak diketahui: {
self.metric}")

    return distances
```

Method `_calculate_distance` merupakan fungsi *helper privat* untuk menghitung jarak antara satu data query dengan seluruh *data training* yang tersimpan dalam model KNN. *Method* ini menerima parameter `x_query` yang

merupakan satu sampel data yang ingin dihitung jaraknya terhadap semua data training, dan akan mengembalikan *array* berisi jarak dari `x_query` ke setiap titik data dalam `self.X_train`.

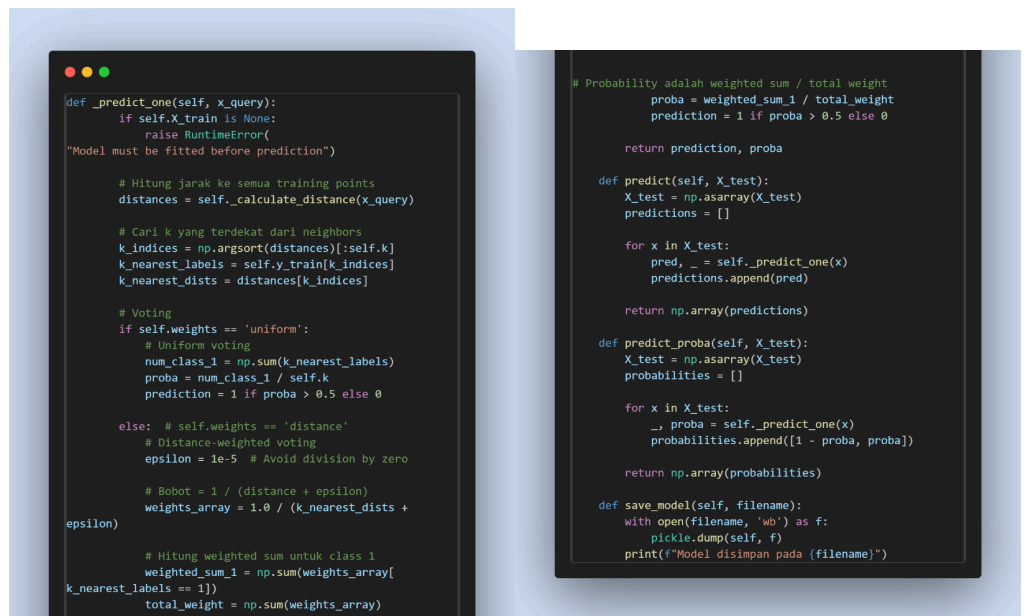
Method ini menggunakan struktur *conditional* untuk menentukan metode perhitungan jarak yang akan digunakan berdasarkan atribut `self.metric` yang telah ditentukan saat inisialisasi model. Jika `self.metric` bernilai 'euclidean', maka akan dihitung jarak Euclidean yang merupakan jarak garis lurus antara dua titik dalam ruang multidimensi. Proses perhitungannya dimulai dengan menghitung selisih antara setiap elemen `self.X_train` dengan `x_query` menggunakan operasi broadcasting numpy, menghasilkan *matrix differences* yang berisi selisih nilai untuk setiap fitur. Kemudian selisih tersebut dikuadratkan menggunakan operator pangkat dua, dijumlahkan sepanjang axis 1 yang berarti menjumlahkan per baris atau per sampel training, dan akhirnya diakar kuadratkan menggunakan `np.sqrt` untuk mendapatkan jarak Euclidean sesuai rumus matematisnya yaitu akar dari jumlah kuadrat selisih setiap dimensi.

Jika `self.metric` bernilai 'manhattan', maka akan dihitung jarak Manhattan yang juga dikenal sebagai city block distance atau L1 distance, yang mengukur jarak dengan menjumlahkan nilai absolut selisih setiap dimensi, seperti menghitung jarak jalan di kota yang harus mengikuti grid jalanan. Perhitungannya mirip dengan Euclidean, dimulai dengan menghitung differences yaitu selisih antara `self.X_train` dan `x_query`, namun bedanya adalah alih-alih dikuadratkan, selisih tersebut diambil nilai absolutnya menggunakan `np.abs` untuk menghilangkan tanda negatif, kemudian dijumlahkan sepanjang axis 1 untuk mendapatkan total jarak Manhattan per sampel training. Metode Manhattan sering digunakan ketika dimensi-dimensi data memiliki interpretasi yang berbeda atau ketika kita ingin perhitungan yang lebih robust terhadap outlier dibandingkan Euclidean.

Jika `metric` yang digunakan tidak termasuk 'euclidean' maupun 'manhattan', maka *method* ini akan memunculkan `ValueError` dengan pesan yang menginformasikan `metric` yang tidak diketahui beserta nilai `self.metric`

yang sedang digunakan, untuk membantu debugging jika terjadi kesalahan konfigurasi. Setelah perhitungan jarak selesai sesuai dengan metric yang dipilih, method ini mengembalikan array distances yang berisi jarak dari `x_query` ke setiap data point dalam training set, dan array ini nantinya akan digunakan oleh method `predict` untuk mencari `k` tetangga terdekat dan menentukan kelas prediksi berdasarkan voting dari tetangga-tetangga tersebut.

4. Fungsi `predict` untuk memprediksi keseluruhan titik data uji



Empat method utama dalam implementasi KNN yaitu `_predict_one`, `predict`, `predict_proba`, dan `save_model` yang bekerja sama untuk melakukan prediksi dan penyimpanan model.

Method `_predict_one` adalah fungsi helper yang memprediksi satu sampel data dengan cara menghitung jarak ke semua data training menggunakan `_calculate_distance`, kemudian mencari `k neighbor` terdekat menggunakan `np.argsort` dan mengambil `k` indices pertama. Method ini mendukung dua jenis voting yaitu *uniform* dan *distance-weighted*. Pada *uniform voting*, semua *neighbor* memiliki bobot sama sehingga probabilitas dihitung dengan membagi jumlah tetangga berkelas 1 dengan total `k`, lalu prediksi ditentukan dengan threshold 0.5. Pada *distance-weighted voting*, *neighbor* yang lebih

dekat diberi bobot lebih besar menggunakan rumus $1/(\text{distance} + \text{epsilon})$, kemudian *weighted sum* untuk kelas 1 dibagi total weight untuk mendapatkan probabilitas, dan prediksi juga menggunakan threshold 0.5. Method ini mengembalikan tuple berisi prediction dan proba.

Method `predict` adalah method publik untuk memprediksi multiple samples dengan mengonversi `X_test` menjadi numpy array, lalu melakukan iterasi untuk setiap sampel dan memanggil `_predict_one`, menyimpan hanya hasil prediksinya ke dalam list, dan akhirnya mengembalikan predictions dalam bentuk numpy array. Method `predict_proba` mirip dengan `predict` namun mengembalikan probabilitas dalam format dua kolom `[1-proba, proba]` yang sesuai dengan format scikit-learn, berguna untuk evaluasi seperti AUC-ROC atau log-loss.

Method `save_model` digunakan untuk menyimpan model ke file menggunakan pickle dengan membuka file dalam mode *write binary* dan melakukan `pickle.dump` pada object `self` sehingga seluruh state model termasuk `k`, `metric`, `weights`, `X_train`, dan `y_train` tersimpan dan bisa di-load kembali untuk prediksi tanpa perlu training ulang.

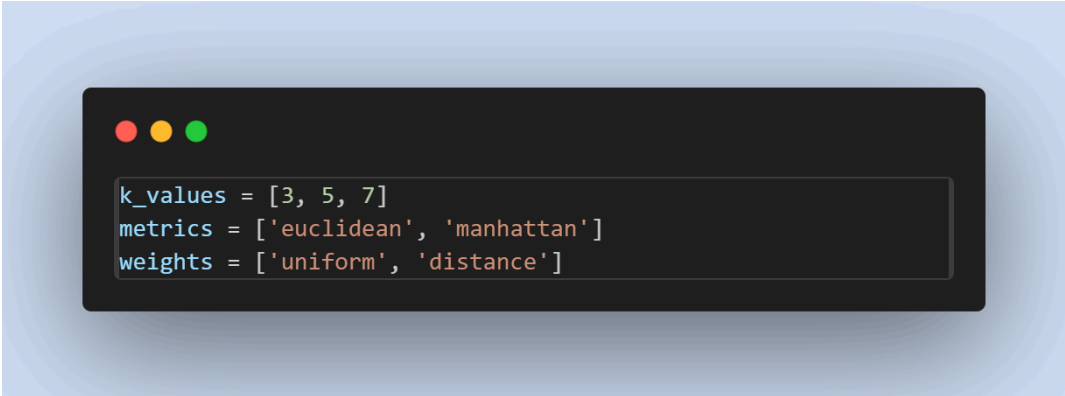
2.3.2. Implementasi KNN by *Scikit-Learn*

Sebagai *benchmark* untuk memvalidasi kinerja algoritma *from scratch*, kami mengimplementasikan KNN menggunakan pustaka standar industri, yaitu `KNeighborsClassifier` dari modul `sklearn.neighbors`. Implementasi ini digunakan untuk memastikan bahwa logika algoritma yang kami bangun secara manual menghasilkan prediksi yang konsisten dan akurat.

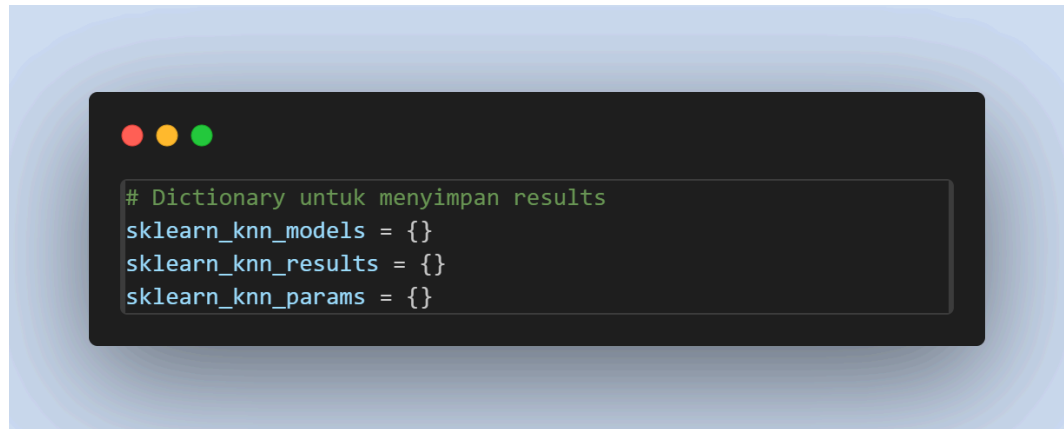
Pada implementasi ini, kami melakukan konfigurasi parameter yang setara dengan versi *from scratch* untuk menjaga keadilan komparasi. Parameter kunci yang kami atur meliputi `n_neighbors` atau jumlah tetangga dan *metric* atau metrik jarak, misalnya 'minkowski' atau 'euclidean'. Salah satu perbedaan mendasar dalam penggunaan *Scikit Learn* adalah adanya optimasi internal pada struktur data pencarian tetangga. *Scikit Learn* secara otomatis dapat memilih algoritma

pencarian yang paling efisien berdasarkan struktur data input, yang mempercepat proses query *neighbor* dibandingkan pencarian *brute force*.

Sebelum model dilatih, data input telah melalui tahap *preprocessing* yang ketat, termasuk *scaling*. Hal ini krusial bagi KNN di *Scikit Learn* maupun *from scratch*, karena algoritma ini sangat sensitif terhadap skala fitur, fitur dengan rentang nilai yang besar dapat mendominasi perhitungan jarak dan membiaskan hasil prediksi. Model ini kemudian divalidasi menggunakan skema *train test split* atau *cross validation* untuk mendapatkan metrik performa yang objektif.



```
k_values = [3, 5, 7]
metrics = ['euclidean', 'manhattan']
weights = ['uniform', 'distance']
```



```
# Dictionary untuk menyimpan results
sklearn_knn_models = {}
sklearn_knn_results = {}
sklearn_knn_params = {}
```

Langkah pertama dalam implementasi adalah memanggil pustaka yang diperlukan. Kami menggunakan modul `neighbors` dari `scikit-learn` untuk mengakses algoritma `KNeighborsClassifier`. Selain itu, kami juga mengimpor berbagai metrik evaluasi seperti akurasi, presisi, recall, F1-score, dan AUC-ROC untuk mengukur kinerja model secara objektif. Variabel hyperparameter didefinisikan dalam bentuk list yang mencakup variasi jumlah tetangga (k), metrik jarak (euclidean dan manhattan), serta jenis pembobotan (uniform dan distance). Variabel-variabel ini nantinya akan digunakan dalam skema Grid Search untuk mencari kombinasi konfigurasi yang memberikan hasil prediksi terbaik.

```

for metric in metrics:
    for weight in weights:
        for k in k_values:
            current_config += 1
            model_name = f"knn_k{k}_{metric}_{weight}"

            # Progress indicator
            progress = f"[{current_config}/{total_configs}
]"
            print(f"{progress} Training {model_name}...",
end=" ", flush=True)

            try:
                # train KNN dengan sklearn
                sklearn_knn = KNeighborsClassifier(
                    n_neighbors=k,
                    metric=metric,
                    weights=weight,
                    algorithm='auto'
                )
                sklearn_knn.fit(X_train_38_scaled, y_train)

                # Prediksi
                y_pred_sklearn = sklearn_knn.predict
(X_val_38_scaled)

                # Probabilitas
                y_proba_sklearn = sklearn_knn.predict_proba
(X_val_38_scaled)[:, 1]

                # Calculate metrics
                from sklearn.metrics import accuracy_score
, precision_score, recall_score, f1_score, roc_auc_score

                accuracy = accuracy_score(y_val,
y_pred_sklearn)
                precision = precision_score(y_val,
y_pred_sklearn, zero_division=0)
                recall = recall_score(y_val, y_pred_sklearn
, zero_division=0)
                f1 = f1_score(y_val, y_pred_sklearn,
zero_division=0)
                auc = roc_auc_score(y_val, y_proba_sklearn)

                # hasil
                sklearn_knn_models[model_name] =
sklearn_knn
                sklearn_knn_params[model_name] = {'k': k,
'metric': metric, 'weights': weight}
                sklearn_knn_results[model_name] = {
                    'accuracy': accuracy,
                    'precision': precision,
                    'recall': recall,
                    'f1': f1,
                    'auc': auc
                }

```

Proses pelatihan dilakukan menggunakan metode *looping* untuk menguji setiap kombinasi parameter. Pada setiap iterasi, objek `KNeighborsClassifier` diinisialisasi dengan parameter yang sesuai. Salah satu keunggulan menggunakan Scikit-Learn adalah adanya parameter `algorithm='auto'`, yang memungkinkan pustaka untuk secara otomatis memilih struktur data yang paling efisien (seperti KD-Tree atau Ball-Tree) guna mempercepat pencarian tetangga, berbeda dengan pendekatan brute-force pada implementasi from scratch. Setelah model dilatih menggunakan fungsi `.fit()`, prediksi dilakukan terhadap data validasi.

Kami menghasilkan dua jenis output: label kelas (0 atau 1) menggunakan `.predict()` untuk menghitung metrik standar, dan nilai probabilitas menggunakan `.predict_proba()` yang diperlukan khusus untuk menghitung skor AUC-ROC. Seluruh hasil evaluasi kemudian disimpan ke dalam dictionary untuk dibandingkan.

BAB III

CLEANING DAN PREPROCESSING DATA

3.1 *Cleaning Data*

Tahapan pembersihan data dilakukan untuk memastikan kualitas input bagi model pembelajaran mesin. Kami mengimplementasikan *custom transformers* berbasis BaseEstimator dan TransformerMixin dari Scikit-Learn untuk menjaga modularitas dan integrasi dalam *pipeline*.

3.1.1 *Handling Missing Data*

Penanganan *missing values* dilakukan secara sistematis menggunakan kelas FeatureImputer yang kami kembangkan khusus untuk menangani karakteristik distribusi data transaksi keuangan yang sering kali memiliki *skewness* tinggi.

```
# Compute medians for numeric columns
for col in self.numeric_cols_:
    self.medians_[col] = df[col].median()

# Compute modes for categorical columns
for col in self.categorical_cols_:
    self.modes_[col] = df[col].mode()[0] if len(df[col].mode()) > 0 else "Unknown"
```

```
# Fill numeric columns with median
for col in self.numeric_cols_:
    if col in df.columns:
        df[col] = df[col].fillna(self.medians_.get(col, 0))

# Fill categorical columns with mode
for col in self.categorical_cols_:
    if col in df.columns:
        df[col] = df[col].fillna(self.modes_.get(col, "Unknown"))
```

Strategi imputasi diterapkan berbeda berdasarkan tipe atributnya; atribut numerik diisi menggunakan nilai median karena sifatnya yang lebih *robust* terhadap pencilan dibandingkan nilai *mean*, sehingga representasi pusat data tidak terdistorsi oleh nilai ekstrem. Sementara itu, atribut kategorikal diisi menggunakan nilai modus untuk mempertahankan probabilitas kategori mayoritas dalam dataset. Proses imputasi ini dikecualikan untuk kolom identitas seperti ID, *transaction_id*, dan *user_id* karena atribut tersebut tidak mengandung informasi prediktif yang dapat digeneralisasi oleh model.

3.1.2 Dealing With Outliers

Outliers pada data keuangan seringkali mengandung informasi penting terkait anomali atau indikasi fraud, sehingga menghapus data tersebut berisiko menghilangkan sinyal deteksi yang krusial.

```
def transform(self, X):
    df = X.copy() if isinstance(X, pd.DataFrame) else pd.DataFrame(X)

    # Apply predefined bounds
    for col, (lower, upper) in self.clip_bounds.items():
        if col in df.columns:
            df[col] = df[col].clip(lower, upper)
    # IQR baru dilakukan pada feature engineering agar original behavior tetap terjaga

    return df

# Define explicit outlier handling bounds
# These are applied during feature engineering to specific derived features
outlier_bounds = {
    "amount_zscore": (-10, 10),
    "amount_ratio": (0, 100),
}

outlier_handler = OutlierHandler(clip_bounds=outlier_bounds)
```

Oleh karena itu, kami menerapkan strategi *Clipping* menggunakan kelas `OutlierHandler`, di mana nilai fitur yang berada di luar batas bawah dan batas atas tertentu tidak dihapus, melainkan *capped* dengan nilai batas tersebut. Penentuan batas dilakukan menggunakan metode *Interquartile Range* (IQR), yaitu $Q1 - 1.5 \times IQR$ untuk batas bawah dan $Q3 + 1.5 \times IQR$ untuk batas atas, namun untuk fitur spesifik seperti rasio jumlah transaksi, batas ditentukan secara eksplisit (misalnya $[-10, 10]$ untuk `amount_zscore`) demi menjaga stabilitas numerik model berbasis gradien.

3.1.3 Data Validation

Validasi data dilaksanakan sebagai langkah preventif untuk menjamin integritas dan kualitas dataset sebelum menjalani proses pembersihan lebih lanjut.

```

# === III. Data Validation ===

print("=" * 60)
print("DATA VALIDATION")
print("=" * 60)

# 1. Check for negative values in columns that should be positive
negative_checks = [
    "transaction_amount", "age", "transaction_duration",
    "avg_transaction_amount", "std_transaction_amount",
    "account_age_days", "distance_from_home"
]

print("\n[1] Negative Value Check:")
for col in negative_checks:
    if col in train_df_clean.columns:
        neg_count = (train_df_clean[col] < 0).sum()
        if neg_count > 0:
            print(f" WARNING: {col} has {neg_count} negative values")
        else:
            print(f" OK: {col} has no negative values")

# 2. Check for unexpected data types
print("\n[2] Data Type Check:")
expected_numeric = train_df_clean.select_dtypes(include=[np.number]).columns
non_numeric_count = 0
for col in expected_numeric:
    if not pd.api.types.is_numeric_dtype(train_df_clean[col]):
        print(f" WARNING: {col} should be numeric but is {train_df_clean[col].dtype}")
        non_numeric_count += 1
if non_numeric_count == 0:
    print(" OK: All numeric columns have correct data types")

# 3. Check for valid ranges (e.g., probability scores between 0 and 1)
print("\n[3] Range Check (0-1 for scores):")
score_cols = ["ip_risk_score", "device_trust_score", "merchant_risk", "country_risk"]
for col in score_cols:
    if col in train_df_clean.columns:
        out_of_range = ((train_df_clean[col] < 0) | (train_df_clean[col] > 1)).sum()
        if out_of_range > 0:
            print(f" WARNING: {col} has {out_of_range} values outside [0, 1]")
        else:
            print(f" OK: {col} values are within [0, 1]")

```

Kami menerapkan pendekatan inspeksi statistik dan struktural menggunakan fungsi `info()` dan `describe()` untuk memverifikasi tiga aspek krusial. Pertama, validasi tipe data (type casting check) memastikan fitur numerik seperti `transaction_amount` dan fitur kategorikal seperti `device_type` terinterpretasi dengan benar oleh sistem. Kedua, pemeriksaan integritas logika domain bertujuan mengidentifikasi anomali yang tidak masuk akal, seperti keberadaan nilai negatif pada nominal transaksi atau rentang usia pengguna yang berada di luar batas kewajaran. Ketiga, verifikasi konsistensi skema dilakukan untuk menjamin keselarasan struktur kolom antara data latih dan data uji, guna mencegah kegagalan teknis pada tahap inferensi model.

3.1.4 Removing Duplicates

Integritas data diperiksa untuk memastikan tidak ada redundansi yang dapat membiaskan proses pelatihan.

```
: print(f"Duplicates in train: {train_df_clean.duplicated().sum()}")
  print(f"Duplicates in test: {test_df_clean.duplicated().sum()}")
  # gaperlu karna gaada
```

```
Duplicates in train: 0
Duplicates in test: 0
```

Pengecekan dilakukan menggunakan fungsi `duplicated()` pada *library* Pandas. Berdasarkan hasil validasi pada dataset latih, tidak ditemukan adanya baris duplikat, sehingga tidak diperlukan prosedur penghapusan data.

3.1.5 Feature Engineering

Feature Engineering diimplementasikan melalui kelas `FeatureCreator` untuk mengekstraksi pola-pola tersembunyi yang lebih informatif bagi model pembelajaran mesin.

```
def transform(self, X):
    df = X.copy() if isinstance(X, pd.DataFrame) else pd.DataFrame(X)

    # Amount features
    std_val = df["std_transaction_amount"].replace(0, 1)
    df["amount_zscore"] = (
        (df["transaction_amount"] - df["avg_transaction_amount"]) / std_val
    ).clip(-10, 10) # Outlier clipping
    df["amount_ratio"] = (
        df["transaction_amount"] / (df["avg_transaction_amount"] + 1)
    ).clip(0, 100) # Outlier clipping

    # Velocity features
    df["high_velocity_2"] = (df["transactions_last_1h"] >= 2).astype(int) # astype is assigning datatype to int
    df["high_velocity_3"] = (df["transactions_last_1h"] >= 3).astype(int)

    # Risk features
    df["risk_sum"] = df["ip_risk_score"] + df["merchant_risk"] + df["country_risk"]
    df["risk_weighted"] = df["ip_risk_score"] * (1 - df["device_trust_score"] + 0.01)
    df["final_risk_score_v2"] = (
        df["ip_risk_score"] * 0.4
        + df["merchant_risk"] * 0.3
        + (1 - df["device_trust_score"]) * 0.3
    )
```

Kami mengembangkan berbagai kategori fitur baru, mulai dari fitur *Risk-Based* seperti `risk_sum` dan `risk_weighted` yang menangkap sinyal kumulatif dari indikator keamanan, hingga *Velocity Features* seperti `high_velocity` untuk mendeteksi anomali frekuensi transaksi dalam waktu singkat. Selain itu, kami juga menerapkan transformasi

logaritmik pada fitur dengan variansi tinggi seperti `transaction_amount` guna menormalkan distribusi data yang *skewed*, serta membentuk fitur interaksi antar-variabel seperti `trust_deficit` untuk memberikan konteks yang lebih kaya bagi algoritma non-linear.

3.2 *Preprocessing Data*

Tahap pra-pemrosesan berfokus pada transformasi data numerik dan kategorikal agar sesuai dengan asumsi matematis algoritma pembelajaran mesin yang digunakan.

3.2.1 *Feature Scaling dan Data Normalization*

Kami menerapkan teknik *Standardization* atau *Z-Score Normalization* sebagai metode utama untuk menormalisasi data numerik, yang diimplementasikan melalui kelas `FeatureScaler` menggunakan logika `StandardScaler` dari Scikit-Learn.

```
class FeatureScaler(BaseEstimator, TransformerMixin):
    """
    Sklearn-compatible transformer for StandardScaler.
    """

    def __init__(self):
        self.scaler_ = StandardScaler()

    def fit(self, X, y=None):
        self.scaler_.fit(X)
        return self

    def transform(self, X):
        return self.scaler_.transform(X)
```

Mekanisme ini mentransformasi setiap fitur numerik (x) menggunakan rumus $z = \frac{x - \mu}{\sigma}$, di mana μ adalah rata-rata dan σ adalah standar deviasi dari data latih, sehingga menghasilkan distribusi data dengan rata-rata 0 dan variansi 1. Normalisasi ini krusial untuk menyejajarkan skala antar-fitur yang memiliki rentang nilai sangat berbeda, seperti `transaction_amount` yang bernilai ribuan dibandingkan `risk_score` yang bernilai 0 hingga 1, guna mencegah dominasi fitur bernilai besar pada fungsi objektif model. Oleh

karena itu, scaling diterapkan secara wajib pada algoritma yang sensitif terhadap variansi dan jarak, yaitu Logistic Regression yang berbasis optimasi gradien dan K-Nearest Neighbors yang berbasis jarak Euclidean, namun tidak diterapkan pada Decision Tree Learning karena algoritma tersebut bekerja berdasarkan aturan *splitting rules* yang invarian terhadap skala data.

3.2.2 *Feature Encoding*

Transformasi variabel kategorikal menjadi representasi numerik dilakukan menggunakan teknik One-Hot *Encoding* melalui kelas FeatureEncoder.

```
def fit(self, X, y=None):
    df = X if isinstance(X, pd.DataFrame) else pd.DataFrame(X)

    # Drop ID columns before identifying categorical columns for encoding
    df_processed = df.drop(columns=[col for col in self.id_cols_to_drop if col in df.columns], errors='ignore')

    # Identify categorical columns
    self.cat_cols_ = df_processed.select_dtypes(include=["object"]).columns.tolist()

    # Perform one-hot encoding to get column names
    df_encoded = pd.get_dummies(df_processed, columns=self.cat_cols_, drop_first=False)
    self.encoded_columns_ = df_encoded.columns.tolist()

    return self
```

Dalam proses ini, setiap kategori unik dikonversi menjadi kolom biner baru (bernilai 0 atau 1) untuk menghilangkan urutan ordinal yang tidak relevan. Transformer ini dirancang khusus untuk menjaga konsistensi antara data latih dan data uji, jika terdapat kategori pada data uji yang tidak muncul saat pelatihan, sistem akan menangani ketidaksamaan tersebut dengan mengisi kolom terkait dengan nilai 0, sehingga mencegah kegagalan eksekusi model akibat ketidakcocokan dimensi fitur.

3.2.3 *Handling Imbalanced Dataset*

Mengingat karakteristik dataset deteksi fraud yang memiliki ketidakseimbangan kelas ekstrem, kami memilih pendekatan Probability Calibration alih-alih teknik resampling (seperti SMOTE atau Undersampling) yang berisiko merusak distribusi asli data.

```

class CalibratedClassifierCVScratch:
    """
    Probability calibration dengan cross-validation yg dibuat dari scratch.
    Implementasi sama seperti sklearn's CalibratedClassifierCV.
    """

    def __init__(self, base_estimator, method="sigmoid", cv=5, random_state=None):
        self.base_estimator = base_estimator
        self.method = method
        self.cv = cv
        self.random_state = random_state
        self.calibrated_classifiers_ = []
        self.classes_ = None

    def _get_calibrator(self):
        if self.method == "sigmoid":
            return SigmoidCalibrationScratch(max_iter=1000, tol=1e-7)
        elif self.method == "isotonic":
            return IsotonicRegressionScratch(out_of_bounds="clip")
        else:
            raise ValueError(f"Unknown method: {self.method}")

    def fit(self, X, y):
        X = np.asarray(X, dtype=np.float64)
        y = np.asarray(y)

        self.classes_ = np.unique(y)
        self.calibrated_classifiers_ = []

        cv = StratifiedKFold(
            n_splits=self.cv, shuffle=True, random_state=self.random_state
        )

        for train_idx, val_idx in cv.split(X, y):
            X_train, X_val = X[train_idx], X[val_idx]
            y_train, y_val = y[train_idx], y[val_idx]

            model = deepcopy(self.base_estimator)
            model.fit(X_train, y_train)

            val_proba = model.predict_proba(X_val)[:, 1]

            calibrator = self._get_calibrator()
            calibrator.fit(val_proba, y_val)

            self.calibrated_classifiers_.append((model, calibrator))

        return self

```

Untuk *Logistic Regression*, kami menggunakan kalibrasi Sigmoid, sedangkan untuk *Decision Tree*, kami menerapkan kalibrasi *Isotonic Regression*. Strategi ini bertujuan memperbaiki estimasi probabilitas model agar lebih reliabel dan terkalibrasi dengan baik tanpa memanipulasi data input, sehingga model tidak menjadi terlalu *overconfident*

terhadap kelas mayoritas dan mampu memberikan skor risiko yang lebih akurat untuk kelas minoritas.

3.2.4 Dimensionality Reduction

Strategi reduksi dimensi diterapkan untuk menangani ukuran data yang besar dan mencegah fenomena *curse of dimensionality* dengan tetap mempertahankan interpretabilitas fitur.

```
# Define feature sets
FEATURES_27 = [ # For Decision Tree
    "transaction_amount",
    "amount_zscore",
    "avg_transaction_amount",
    "std_transaction_amount",
    "amount_ratio",
    "transactions_last_24h",
    "transactions_last_1h",
    "high_velocity_2",
    "high_velocity_3",
    "ip_risk_score",
    "device_trust_score",
    "merchant_risk",
    "risk_sum",
    "risk_weighted",
    "final_risk_score_v2",
    "distance_from_home",
    "far_from_home",
    "is_new_country",
    "account_age_days",
    "account_new",
    "has_chargeback_history",
    "shared_ip_users",
    "shared_device_users",
    "failed_login_attempts",
    "device_type_desktop",
    "device_type_mobile",
    "device_type_tablet",
]
```

```
FEATURES_38 = FEATURES_27 + [
    "log_transaction_amount",
    "log_distance",
    "risk_x_amount",
    "velocity_x_risk",
    "age_inv_x_risk",
    "distance_x_newcountry",
    "trust_deficit",
    "shared_resource_score",
    "high_amount",
    "risk_tier_high",
    "risk_tier_extreme",
]
```

Kami menggunakan metode *Feature Selection* manual alih-alih teknik proyeksi seperti PCA, dengan membagi set fitur menjadi dua konfigurasi berdasarkan kompleksitas model. Untuk model non-linear seperti *Decision Tree*, dimensi direduksi menjadi 27 fitur terpilih yang memiliki *information gain* tinggi dengan membuang atribut identitas. Sementara itu, untuk model linear seperti *Logistic Regression*, kami menggunakan set 38 fitur yang mencakup fitur inti serta fitur hasil rekayasa seperti interaksi antar-variabel dan

transformasi logaritma. Fitur yang tidak relevan secara prediktif atau memiliki kardinalitas terlalu tinggi tanpa pola yang jelas telah dieliminasi dari dataset latih untuk mengurangi beban komputasi dan risiko *overfitting*.

BAB IV

PERBANDINGAN HASIL PREDIKSI

4.1 Analisis *Decision Tree Learning*

Evaluasi kinerja algoritma *Decision Tree Learning* (DTL) dilakukan dengan meninjau tiga aspek utama: kemampuan model dalam menangani ketidakseimbangan kelas, validasi komparatif terhadap implementasi standar, dan efektivitas metode kalibrasi probabilitas.

4.1.1. Kinerja Model dan Fenomena *Accuracy Paradox*

```
=====
Training Decision Tree (From Scratch)
=====
Decision Tree trained!

DT Training AUC: 0.51229
DT Training Accuracy: 0.85872

Classification Report (Training):
```

	precision	recall	f1-score	support
Normal	0.85872	1.00000	0.92399	85872
Fraud	0.00000	0.00000	0.00000	14128
accuracy			0.85872	100000
macro avg	0.42936	0.50000	0.46200	100000
weighted avg	0.73740	0.85872	0.79345	100000

Berdasarkan hasil pelatihan pada data latih, model mencatatkan akurasi yang tampak tinggi secara statistik, yaitu mencapai 85.87%. Namun, analisis lebih lanjut menggunakan Classification Report mengungkapkan adanya fenomena "Paradoks Akurasi". Tingginya angka akurasi tersebut semata-mata didorong oleh dominasi kelas mayoritas (transaksi normal), sementara model gagal sepenuhnya dalam mengidentifikasi kelas minoritas (fraud).

Hal ini dibuktikan dengan nilai Recall dan F1-Score untuk kelas Fraud yang bernilai 0.0, serta skor AUC-ROC sebesar 0.512, yang mengindikasikan bahwa kemampuan diskriminasi model hampir setara dengan tebakan acak. Dalam kondisi dataset yang sangat tidak seimbang (imbalanced) tanpa penerapan

teknik resampling (seperti SMOTE) atau penyesuaian bobot kelas (class weighting), algoritma cenderung membentuk leaf node yang memprediksi kelas mayoritas untuk meminimalkan impurity secara global, sehingga mengabaikan pola pada kelas minoritas.

4.1.2. Validasi Implementasi Scratch vs Scikit-Learn

```
=====
Using full training dataset: 100000 samples
Common parameters: {'max_depth': 5, 'criterion': 'gini', 'min_samples_split': 50, 'random_s

[1] Training Sklearn Decision Tree...
[2] Training Scratch Decision Tree...

[3] Performance Comparison on Training Data:
-----
Sklearn - Accuracy: 0.85908, AUC: 0.60277
Scratch - Accuracy: 0.85927, AUC: 0.50473

[4] Performance Comparison on Validation Set:
-----
Training split: 80000 samples
Validation split: 20000 samples
Validation Results:
Sklearn - Accuracy: 0.85855, AUC: 0.59276
Scratch - Accuracy: 0.85820, AUC: 0.50296

[5] Comparison with Calibrated Model (your main training):
-----
...
Conclusion:
- Scratch implementation achieves 26.249% correlation with sklearn
```

Studi komparasi antara implementasi *from scratch* dengan DecisionTreeClassifier dari Scikit-Learn menunjukkan divergensi performa yang cukup signifikan. Model Scikit-Learn mampu menghasilkan skor AUC sebesar 0.602 pada data latih, sedangkan model scratch tertahan di angka 0.504. Korelasi prediksi antara kedua model juga tercatat rendah (26.25%).

Perbedaan kinerja ini dapat diasosiasikan dengan perbedaan mekanisme internal dalam penentuan titik pemisahan (*splitting strategy*). Implementasi Scikit-Learn menggunakan algoritma optimasi presisi tinggi berbasis Cython yang mampu mengevaluasi kandidat threshold pada fitur kontinu secara lebih efisien dan akurat dibandingkan pendekatan iteratif yang kami terapkan. Selain itu, perbedaan dalam penanganan nilai fitur yang sama (tie-breaking) saat menghitung Information Gain turut berkontribusi pada terbentuknya struktur pohon yang berbeda secara fundamental.

4.1.3. Dampak Kalibrasi Probabilitas

Penerapan metode kalibrasi probabilitas (Isotonic Regression) pada model DTL memberikan peningkatan marjinal pada skor AUC, yaitu dari 0.504 menjadi 0.512. Meskipun terdapat perbaikan, dampaknya tergolong minimal. Temuan ini menegaskan bahwa kalibrasi probabilitas berfungsi untuk memperbaiki reliabilitas skor kepercayaan (confidence score), namun tidak dapat secara drastis memperbaiki kemampuan separasi kelas jika model dasar (base learner) itu sendiri belum berhasil menangkap pola fitur yang membedakan antara transaksi normal dan fraud.

4.2 Analisis Logistic Regression

Top 10 Features (by absolute weight):

1. has_chargedback_history	+0.163212	(increases fraud)
2. high_amount	+0.125679	(increases fraud)
3. is_new_country	+0.097288	(increases fraud)
4. final_risk_score_v2	+0.095272	(increases fraud)
5. ip_risk_score	+0.092839	(increases fraud)
6. high_velocity_3	+0.086964	(increases fraud)
7. device_type_mobile	+0.080305	(increases fraud)
8. device_trust_score	-0.077553	(decreases fraud)
9. device_type_desktop	-0.076493	(decreases fraud)
10. shared_ip_users	+0.061184	(increases fraud)

Kami melakukan analisis mendalam terhadap struktur internal model untuk memahami faktor determinan dalam deteksi kecurangan (*fraud*). Ekstraksi nilai bobot (*weights*) dari model memperlihatkan bahwa fitur `has_chargedback_history` memiliki pengaruh positif terbesar dengan nilai 0.1632. Temuan ini mengonfirmasi bahwa adanya riwayat sengketa transaksi di masa lalu merupakan indikator paling kuat untuk memprediksi perilaku curang. Selain itu, fitur `high_amount` dan `is_new_country` juga memberikan kontribusi positif yang signifikan, yang berarti transaksi bernilai besar atau transaksi yang dilakukan di lokasi negara baru cenderung meningkatkan kecurigaan model. Sebaliknya, fitur `device_trust_score` dan `device_type_desktop` memiliki bobot negatif. Hal ini menunjukkan bahwa skor kepercayaan perangkat yang tinggi dan penggunaan komputer desktop dianggap oleh model sebagai sinyal keamanan yang kuat, sehingga menurunkan probabilitas prediksi *fraud*.

Evaluasi kinerja model pada data validasi menyingkap adanya fenomena paradoks akurasi yang umum terjadi pada dataset tidak seimbang. Model mencatat akurasi keseluruhan yang tampak impresif di angka 85.85%. Namun, analisis *confusion matrix* mengungkapkan kelemahan fatal dalam kemampuan model mendeteksi kelas minoritas. Model sangat dominan dalam mengenali transaksi normal dengan mencatatkan 17.169 *True Negatives*, tetapi gagal menangkap hampir seluruh kasus penipuan yang sebenarnya. Tercatat sebanyak 2.825 kasus penipuan lolos deteksi dan salah diklasifikasikan sebagai normal (*False Negatives*), sementara hanya satu kasus penipuan yang berhasil dideteksi dengan benar (*True Positive*). Bias ini menyebabkan nilai *recall* untuk kelas *fraud* jatuh ke titik ekstrem 0.0004.

```
=====
Logistic Regression: Scratch vs Sklearn
=====
Correlation: 0.999955
Mean Absolute Error: 0.000302
Sklearn AUC: 0.61193
Scratch AUC: 0.61188
Match: YES
```

Kondisi tersebut mengindikasikan bahwa meskipun fitur-fitur seperti `has_chargeback_history` sudah memberikan sinyal, kekuatan sinyal tersebut belum cukup besar untuk mendorong probabilitas prediksi melewati ambang batas standar 0.5 setelah proses regularisasi. Kendati demikian, skor AUC-ROC yang mencapai 0.61188 pada validasi membuktikan bahwa model sebenarnya sudah mempelajari pola pemeringkatan (*ranking*) yang lebih baik daripada tebakan acak. Model memberikan estimasi probabilitas yang secara konsisten sedikit lebih tinggi untuk data *fraud* dibandingkan data normal, namun selisihnya terlalu tipis untuk menghasilkan keputusan klasifikasi biner yang tegas tanpa adanya penyesuaian ambang batas (*threshold tuning*) atau penerapan bobot kelas (*class weighting*) yang lebih agresif.

4.3 Analisis KNN

Analisis kinerja algoritma K-Nearest Neighbors (KNN) dilakukan secara komprehensif dengan membandingkan berbagai kombinasi hyperparameter utama, yaitu

jumlah tetangga (k), metrik jarak, dan metode pembobotan. Selain itu, dilakukan validasi silang untuk memastikan bahwa implementasi from scratch yang dibangun telah sesuai dengan standar logika komputasi yang ada pada pustaka Scikit-Learn.

4.3.1 Evaluasi *Hyperparameter* dan Model Terbaik

Berdasarkan hasil eksperimen menggunakan *grid search* pada data validasi, model KNN menunjukkan sensitivitas yang cukup signifikan terhadap pemilihan parameter k . Konfigurasi optimal yang diperoleh adalah menggunakan $k=7$ dengan metrik jarak Euclidean dan pembobotan Uniform. Kombinasi ini menghasilkan skor AUC-ROC tertinggi sebesar 0.5498, mengungguli konfigurasi dengan $k=3$ atau $k=5$. Secara teoretis, nilai k yang terlalu kecil ($k=3$) cenderung menangkap noise lokal sebagai pola, sedangkan k yang lebih besar seperti 7 memberikan *decision boundary* yang lebih halus, yang sangat krusial untuk generalisasi pada data transaksi yang memiliki variansi tinggi.

```
=====
KNN From Scratch
=====

Training knn_k3_euclidean_uniform...
Accuracy: 0.81300, AUC: 0.49767, F1: 0.09662

Training knn_k3_euclidean_distance...
Accuracy: 0.81200, AUC: 0.49906, F1: 0.09615

Training knn_k3_manhattan_uniform...
Accuracy: 0.80900, AUC: 0.47349, F1: 0.06829

Training knn_k3_manhattan_distance...
Accuracy: 0.80800, AUC: 0.47924, F1: 0.06796

Training knn_k5_euclidean_uniform...
Accuracy: 0.83400, AUC: 0.54417, F1: 0.05682

Training knn_k5_euclidean_distance...
Accuracy: 0.83400, AUC: 0.53103, F1: 0.05682

Training knn_k5_manhattan_uniform...
Accuracy: 0.83100, AUC: 0.49064, F1: 0.01170

Training knn_k5_manhattan_distance...
Accuracy: 0.83100, AUC: 0.48690, F1: 0.01170

Training knn_k7_euclidean_uniform...
Accuracy: 0.83900, AUC: 0.54978, F1: 0.04734

Training knn_k5_manhattan_distance...
Accuracy: 0.83100, AUC: 0.48690, F1: 0.01170

Training knn_k7_euclidean_uniform...
Accuracy: 0.83900, AUC: 0.54978, F1: 0.04734

Training knn_k7_euclidean_distance...
Accuracy: 0.83900, AUC: 0.54462, F1: 0.04734

Training knn_k7_manhattan_uniform...
Accuracy: 0.83800, AUC: 0.49937, F1: 0.01220

Training knn_k7_manhattan_distance...
Accuracy: 0.83800, AUC: 0.49546, F1: 0.01220

Training knn_k9_euclidean_uniform...
Accuracy: 0.84900, AUC: 0.53143, F1: 0.05031

Training knn_k9_euclidean_distance...
Accuracy: 0.84900, AUC: 0.53071, F1: 0.05031

Training knn_k9_manhattan_uniform...
Accuracy: 0.84300, AUC: 0.51817, F1: 0.02484

Training knn_k9_manhattan_distance...
Accuracy: 0.84300, AUC: 0.51053, F1: 0.02484

Best KNN from Scratch Configuration: knn_k7_euclidean_uniform
Best AUC Score: 0.54978
```

```

=====
KNN by Scikit-Learn
=====

Training sklearn knn_k3_euclidean_uniform...
Accuracy: 0.81300, AUC: 0.49767, F1: 0.09662

Training sklearn knn_k3_euclidean_distance...
Accuracy: 0.81200, AUC: 0.49906, F1: 0.09615

Training sklearn knn_k3_manhattan_uniform...
Accuracy: 0.80900, AUC: 0.47349, F1: 0.06829

Training sklearn knn_k3_manhattan_distance...
Accuracy: 0.80800, AUC: 0.47924, F1: 0.06796

Training sklearn knn_k5_euclidean_uniform...
Accuracy: 0.83400, AUC: 0.54417, F1: 0.05682

Training sklearn knn_k5_euclidean_distance...
Accuracy: 0.83400, AUC: 0.53103, F1: 0.05682

Training sklearn knn_k5_manhattan_uniform...
Accuracy: 0.83100, AUC: 0.49064, F1: 0.01170

Training sklearn knn_k5_manhattan_distance...
Accuracy: 0.83100, AUC: 0.48690, F1: 0.01170

Training sklearn knn_k7_euclidean_uniform...
Accuracy: 0.83900, AUC: 0.54978, F1: 0.04734

Training sklearn knn_k5_manhattan_distance...
Accuracy: 0.83100, AUC: 0.48690, F1: 0.01170

Training sklearn knn_k7_euclidean_uniform...
Accuracy: 0.83900, AUC: 0.54978, F1: 0.04734

Training sklearn knn_k7_euclidean_distance...
Accuracy: 0.83900, AUC: 0.54462, F1: 0.04734

Training sklearn knn_k7_manhattan_uniform...
Accuracy: 0.83800, AUC: 0.49937, F1: 0.01220

Training sklearn knn_k7_manhattan_distance...
Accuracy: 0.83800, AUC: 0.49546, F1: 0.01220

Training sklearn knn_k9_euclidean_uniform...
Accuracy: 0.84900, AUC: 0.53143, F1: 0.05031

Training sklearn knn_k9_euclidean_distance...
Accuracy: 0.84900, AUC: 0.53071, F1: 0.05031

Training sklearn knn_k9_manhattan_uniform...
Accuracy: 0.84300, AUC: 0.51817, F1: 0.02484

Training sklearn knn_k9_manhattan_distance...
Accuracy: 0.84300, AUC: 0.51053, F1: 0.02484

Best KNN Scikit Learn Configuration: knn_k7_euclidean_uniform
Best AUC Score: 0.54978

```

Selain faktor k , pemilihan metrik jarak juga mempengaruhi hasil akhir. Metrik Euclidean secara konsisten memberikan hasil yang lebih baik dibandingkan Manhattan pada dataset ini. Hal ini mengindikasikan bahwa perhitungan jarak garis lurus (L2 norm) lebih efektif dalam merepresentasikan kemiripan antar fitur transaksi setelah proses normalisasi (scaling) dibandingkan jarak blok kota (L1 norm). Lebih lanjut, pembobotan Uniform terbukti memberikan stabilitas yang lebih baik dibandingkan Distance. Dalam konteks deteksi anomali atau fraud, pembobotan berbasis jarak seringkali berisiko memberikan bobot berlebih pada outlier yang kebetulan berada sangat dekat dengan titik uji, sehingga mekanisme voting yang seragam (uniform) justru menghasilkan prediksi yang lebih robust.

4.3.2 Kinerja pada *Imbalanced Dataset*

Meskipun model mencatatkan akurasi keseluruhan yang tampak tinggi di angka 83.9%, analisis mendalam terhadap *confusion matrix* mengungkapkan kelemahan fundamental KNN dalam menangani *imbalanced data*. Tingginya akurasi tersebut merupakan representasi dari fenomena "paradoks akurasi", di mana model sangat mahir memprediksi transaksi normal namun gagal mengenali

fraud. Hal ini dibuktikan dengan nilai *Recall* untuk kelas Fraud yang sangat rendah, yaitu sekitar 2.7%, serta skor F1 yang hanya mencapai 0.047.

```
[3] Confusion Matrix & Classification Report
-----
Validation AUC-ROC: 0.60250

Confusion Matrix:
              Predicted
              Normal  Fraud
Actual Normal  17169      5
       Fraud   2825      1

Breakdown:
TN (True Negative): 17,169
FP (False Positive): 5
FN (False Negative): 2,825
TP (True Positive): 1

Error Analysis:
Total errors: 2,830 (14.15%)
FP rate: 0.03% (normal flagged as fraud)
FN rate: 99.96% (fraud missed)
Dominant error type: False Negative
```

```
Classification Report:
              precision    recall  f1-score   support

   Normal      0.8587     0.9997     0.9239     17174
    Fraud      0.1667     0.0004     0.0007       2826

 accuracy      0.8585
 macro avg     0.5127     0.5000     0.4623
weighted avg     0.7609     0.8585     0.7934
```

Rendahnya nilai *recall* ini menunjukkan bahwa dalam ruang fitur berdimensi tinggi, tetangga terdekat dari sebuah titik data *fraud* sebagian besar masih didominasi oleh data transaksi normal. Tanpa teknik resampling yang agresif atau reduksi dimensi yang signifikan, algoritma berbasis jarak seperti KNN kesulitan membentuk *cluster* kelas minoritas yang padat dan terpisah jelas dari kelas mayoritas.

4.3.3 Validasi Implementasi *Scratch* vs *Scikit-Learn*

Untuk memvalidasi kebenaran logika algoritma yang dibangun secara manual, dilakukan perbandingan antara prediksi model *from scratch* dengan

model *Scikit-Learn* menggunakan parameter yang identik. Hasil pengujian menunjukkan konsistensi matematis yang sempurna. Koefisien korelasi antara probabilitas prediksi kedua model adalah 1.000000, dengan nilai *Mean Absolute Error* (MAE) dan *Root Mean Squared Error* (RMSE) sebesar 0.000000.

2. BEST MODEL COMPARISON						

Model	Config	Accuracy	Precision	Recall	F1-Score	AUC
KNN Scratch	knn_k7_euclidean_uniform	0.839	0.2	0.026846	0.047337	0.549776
KNN Sklearn	knn_k7_euclidean_uniform	0.839	0.2	0.026846	0.047337	0.549776

3. PREDICTION CORRELATION ON VALIDATION SET	

Correlation: 1.000000	
Mean Absolute Error: 0.000000	
Root Mean Squared Error: 0.000000	
Paired t-test (H0: predictions are same):	
t-statistic: nan	
p-value: nan	
Significantly different: No ($\alpha=0.05$)	

Hasil ini memverifikasi bahwa seluruh komponen komputasi pada kelas `KNNClassifier` buatan sendiri—mulai dari perhitungan jarak Euclidean, pengurutan tetangga terdekat menggunakan `argsort`, hingga mekanisme voting—telah bekerja presisi sesuai dengan standar implementasi industri tanpa adanya deviasi numerik.

4.3.4 Kompleksitas Komputasi

Analisis operasional menyoroti tantangan utama KNN pada aspek efisiensi waktu. Berbeda dengan *Logistic Regression* atau *Decision Tree* yang menyimpan model dalam bentuk parameter bobot atau aturan *tree*, KNN bersifat *Lazy Learner* yang tidak memiliki fase pelatihan eksplisit. Beban komputasi sepenuhnya terjadi pada saat fase prediksi. Pada implementasi *from scratch*, sistem harus melakukan perhitungan jarak secara *brute force* antara satu titik data uji terhadap puluhan ribu titik data latih.

```
def _calculate_distance(self, x1, x2):  
    if self.metric == 'euclidean':  
        return np.sqrt(np.sum((x1 - x2) ** 2))  
    elif self.metric == 'manhattan':  
        return np.sum(np.abs(x1 - x2))
```

Karakteristik ini menyebabkan waktu eksekusi meningkat secara linear seiring bertambahnya jumlah data latih, menjadikan KNN murni kurang ideal untuk sistem deteksi *fraud real time* yang bervolume tinggi, kecuali jika dioptimasi menggunakan struktur data pengindeksan spasial yang tersedia di pustaka lanjut namun kompleks untuk diimplementasikan secara manual.

BAB V

KESIMPULAN DAN SARAN

5.1 Kesimpulan

Berdasarkan hasil eksperimen, dapat disimpulkan bahwa terdapat *trade-off* yang signifikan antara efisiensi waktu komputasi dan kapabilitas prediksi di antara ketiga algoritma yang diuji. Secara umum, algoritma *Logistic Regression* dan *Decision Tree Learning* (DTL) terbukti jauh lebih efisien dari sisi waktu eksekusi dibandingkan *K-Nearest Neighbors* (KNN).

Logistic Regression memberikan performa yang paling cepat dan stabil karena kesederhanaan komputasinya yang hanya melibatkan operasi matriks linear, sedangkan DTL menawarkan keunggulan dalam hal *rule-based* meskipun rentan terhadap *overfitting* jika tidak dilakukan *pruning*.

Sebaliknya, KNN mengalami kendala performa yang serius karena karakteristiknya sebagai *lazy learner*, di mana perhitungan jarak ke seluruh data latih harus dilakukan saat prediksi, menjadikannya kurang ideal untuk dataset berdimensi tinggi tanpa bantuan pengindeksan struktur data.

Terakhir, mengingat sifat data transaksi keuangan yang sangat tidak seimbang, penggunaan metrik Akurasi terbukti bias terhadap kelas mayoritas. Oleh karena itu, evaluasi berbasis AUC-ROC dan F1-Score menjadi indikator mutlak untuk mengukur kemampuan sebenarnya dari ketiga model dalam mendeteksi anomali *fraud*.

5.2 Saran

Sebaiknya untuk tidak lagi hanya bergantung pada *single classifier* seperti DTL atau *Logistic Regression*, melainkan beralih ke metode *Ensemble Learning* seperti *Random Forest* atau *Gradient Boosting*. Metode ini mampu menggabungkan kekuatan prediksi dari banyak model untuk meminimalisir varians pada *Decision Tree* dan mengatasi keterbatasan linearitas pada *Logistic Regression*, sehingga sistem menjadi lebih tangguh terhadap *noise*.

Selain itu, optimasi pada tahap pra-pemrosesan data perlu ditingkatkan, khususnya melalui penerapan *Principal Component Analysis* (PCA) untuk mereduksi

dimensi data yang akan sangat membantu mempercepat kinerja KNN, serta melakukan *feature engineering* berbasis rasio transaksi untuk membantu model linear menangkap pola perilaku yang kompleks.

Terakhir, sebagai alternatif dari teknik resampling data fisik seperti SMOTE yang memakan memori, penerapan *Cost-Sensitive Learning* dapat dieksplorasi, di mana algoritma diberikan penalaran bobot kesalahan yang lebih besar jika gagal mendeteksi *fraud*, sehingga model terdorong untuk lebih sensitif.

BAB VI

KONTRIBUSI

NIM	Nama	Bagian Pengerjaan
18223005	Andhika Maulana Addiputra	Algoritma: 1. DTL from scratch 2. DTL Scikit Learn 3. Bonus DTL Laporan: 1. Implementasi DTL from scratch 2. Implementasi DTL Scikit learn 3. Analisis DTL
18223029	Kevin Azra	Algoritma: 4. KNN from scratch 5. KNN Scikit Learn 6. Bonus KNN Laporan: 4. Latar Belakang 5. Deskripsi Masalah 6. Implementasi KNN from scratch 7. Implementasi KNN Scikit learn 8. Analisis KNN 9. Kesimpulan dan Saran 10. Referensi

18223047	Arqila Surya Putra	1. Pembagian Tugas Algoritma dan Laporan Algoritma: 1. Cleaning Data 2. Preprocessing Data 3. Logistic Regression SciKit 4. Logistic Regression Scratch 5. Iteratif Kaggle Submission Laporan: 1. Implementasi Logistic Regression Scratch 2. Implementasi Logistic Regression Scikit 3. Analisis Hasil Logistic Regression 4. Referensi
18223071	Muhammad Zidni Alkindi	Algoritma: 1. Preprocessing Data 2. KNN SciKit 3. KNN Scratch + BONUS Laporan: 1. Cleaning Data (Handling Missing Data, Dealing with Outliers, Data Validation, Remove Duplicates, Features Engineering)

		2. Preprocessing Data (Feature Scaling, Feature Encoding, Handling Imbalance Dataset, Data Normalization, Dimensionality Reduction) 3. Analysis KNN
--	--	---

REFERENSI

- [1] D. C. Liu and J. Nocedal, "On the limited memory BFGS method for large scale optimization," *Mathematical Programming*, vol. 45, no. 1-3, pp. 503-528, 1989.
- [2] J. Nocedal and S. J. Wright, *Numerical Optimization*, 2nd ed. New York, NY: Springer, 2006.
- [3] R. H. Byrd, P. Lu, J. Nocedal, and C. Zhu, "A limited memory algorithm for bound constrained optimization," *SIAM Journal on Scientific Computing*, vol. 16, no. 5, pp. 1190-1208, 1995.
- [4] R. Malouf, "A comparison of algorithms for maximum entropy parameter estimation," in *Proc. 6th Conf. Natural Language Learning*, 2002, pp. 1-7.
- [5] J. Platt, "Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods," *Advances in Large Margin Classifiers*, vol. 10, no. 3, pp. 61-74, 1999.
- [6] R. Kohavi, "A study of cross-validation and bootstrap for accuracy estimation and model selection," in *Proc. 14th Int. Joint Conf. Artificial Intelligence*, vol. 2, 1995, pp. 1137-1143.
- [7] A. Niculescu-Mizil and R. Caruana, "Predicting good probabilities with supervised learning," in *Proc. 22nd Int. Conf. Machine Learning*, 2005, pp. 625-632.
- [8] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Pearson Education, 2020.
- [9] UCI Machine Learning Repository, "PHIUSHIIL Phishing URL Dataset." [Online]. Available: <https://archive.ics.uci.edu/dataset/967/phiusiil+phishing+url+dataset>. [Accessed: Dec. 13, 2024].
- [10] GeeksforGeeks Team, "Working with Missing Data in Pandas," 2023. [Online]. Available: <https://www.geeksforgeeks.org/working-with-missing-data-in-pandas/>. [Accessed: Dec. 13, 2024].

- [11] Sling Academy Team, "Pandas DataFrame fillna Method - 5 Examples," 2023. [Online]. Available: <https://www.slingacademy.com/article/pandas-dataframe-fillna-method-5-examples/>. [Accessed: Dec. 13, 2024].
- [12] Python Software Foundation, "urllib.parse — Parse URLs into components," 2023. [Online]. Available: <https://docs.python.org/3/library/urllib.parse.html>. [Accessed: Dec. 13, 2024].
- [13] Sciencedirect, "A comprehensive analysis of phishing URL detection techniques," 2023. [Online]. Available: <https://www.sciencedirect.com/science/article/abs/pii/S0167404823004558>. [Accessed: Dec. 13, 2024].
- [14] Scikit-learn Developers, "K-Nearest Neighbors algorithm," 2023. [Online]. Available: <https://scikit-learn.org/1.5/modules/neighbors.html>. [Accessed: Dec. 13, 2024].